

INSTITUT SUPÉRIEUR INDUSTRIEL DE BRUXELLES

HAUTE ÉCOLE ISIB

Département Ingénieur Industriel

Finalité Électricité & Automatisation

Travail de Fin d'Études

Présenté en vue de l'obtention du grade de

Master en Sciences de l'Ingénieur Industriel

Orientation Électricité & Automatisation

Localisation autonome de drone en environnement sans GPS

Fusion adaptative vision-inertie sur plateforme X500 V6

Projet GRAND — RMA/VUB

Promoteur :

Nadir BAIBOUN

Présenté par :

Mourad AARAB

Maître de stage :

Joseph MIMASSI — RMA/VUB

Entreprise d'accueil :

Académie Royale Militaire (RMA)

Vrije Universiteit Brussel (VUB)

Avenue de la Renaissance 30, 1000

Bruxelles

Classification : Tout public

Année académique 2025–2026

Résumé

Ce travail traite d'un problème concret : comment faire voler un drone de manière autonome quand le GPS n'est pas disponible ? C'est le cas dans beaucoup de situations réelles — à l'intérieur d'un bâtiment, sous un pont, dans un environnement militaire avec brouillage électronique. Sans GPS, le drone n'a plus de référence de position et devient très difficile à contrôler.

Pour répondre à ce problème, j'ai développé un système de localisation embarqué qui combine deux sources d'information : d'un côté, une caméra OAK-D Pro qui calcule l'odométrie visuelle-inertielle (VIO) directement sur sa puce MyriadX ; de l'autre, l'IMU du contrôleur de vol Pix32 v6C qui fournit les accélérations et vitesses angulaires à 100 Hz. L'idée clé est de ne pas faire confiance aveuglément à la caméra — sa qualité varie selon l'éclairage, la texture des surfaces, le flou de mouvement. J'ai donc ajouté un score de qualité d'image calculé en temps réel, qui ajuste dynamiquement le poids accordé à chaque source dans un filtre de Kalman étendu. Quand l'image se dégrade, le système bascule progressivement vers l'estimation inertielle, sans saut brusque de position.

Le tout tourne sur un drone Holybro X500 V6 avec un ordinateur compagnon Intel UP4000, dans le cadre du projet GRAND (RMA/VUB). Les tests au banc ont confirmé une stabilité de position à ± 5 cm au repos, une synchronisation caméra-IMU à 2,32 ms, et un armement réussi du contrôleur de vol. L'architecture logicielle — 12 nœuds ROS 2 Jazzy répartis en 4 packages — est entièrement open source.

Mots-clés : odométrie visuelle-inertielle, navigation autonome, fusion adaptative de capteurs, drone, GPS-denied, PX4, ROS 2, BasaltVIO, filtre de Kalman étendu

Abstract

This work addresses a practical problem : how to fly a drone autonomously when GPS is unavailable — indoors, under bridges, or in environments with electronic jamming. Without GPS, the flight controller loses its position reference and stable flight becomes very difficult.

The proposed system fuses two on-board sensing modalities. Visual-Inertial Odometry (VIO), computed directly on the MyriadX chip of an OAK-D Pro stereo camera, provides position estimates at 30 Hz. The IMU of the Pix32 v6C flight controller runs at 100 Hz and handles the high-frequency dynamics. The central contribution is an adaptive Extended Kalman Filter whose measurement noise matrix $\mathbf{R}$ is adjusted in real time based on a composite image quality score derived from brightness, sharpness, feature density and contrast. When visual conditions degrade, the system gradually falls back to inertial dead reckoning — without the abrupt position jumps that would destabilise flight.

The system was deployed on a Holybro X500 V6 platform with an Intel UP4000 companion computer. Bench tests confirmed position stability below ± 5 cm at rest, camera-IMU synchronisation at 2.32 ms, and successful flight controller arming. The software stack — 12 ROS 2 Jazzy nodes across 4 packages — is fully open source.

Keywords : visual-inertial odometry, autonomous navigation, adaptive sensor fusion, UAV, GPS-denied, PX4, ROS 2, BasaltVIO, extended Kalman filter

Remerciements

Je tiens à exprimer ma gratitude envers toutes les personnes qui ont contribué à la réalisation de ce travail.

À mon promoteur, pour sa disponibilité et ses conseils qui m’ont permis de garder le cap sur les objectifs essentiels lorsque la complexité technique menaçait de m’éloigner du sujet.

À l’équipe du projet GRAND à l’Académie Royale Militaire et à la VUB, pour m’avoir accueilli et fourni l’accès au matériel et aux laboratoires nécessaires à ce projet. Travailler aux côtés de chercheurs en robotique a été une expérience formatrice dont je mesure la valeur.

À mes collègues de l’ISIB, avec qui les discussions informelles autour d’un café ont souvent débloqué des problèmes que des heures de débogage n’avaient pas résolus. L’ingénierie est un sport d’équipe, même quand le TFE est individuel.

À ma famille, pour leur soutien inconditionnel durant ces années d’études. Leur patience face à mes explications enthousiastes sur les quaternions et les filtres de Kalman — sujets qui ne passionnent pas forcément tout le monde — témoigne d’un amour sans faille.

Enfin, à la communauté open source PX4 et ROS 2, dont les contributions rendent possible des projets comme celui-ci. Un merci particulier aux développeurs de Luxonis et de MAVROS, dont la documentation et les forums m’ont sauvé plus d’une fois à trois heures du matin.

Table des matières

Résumé	i
Abstract	ii
Remerciements	iii
Liste des acronymes	xi
1 Introduction	1
1.1 Contexte et problématique	1
1.2 Objectifs	2
1.3 Ce que ce travail apporte	2
1.4 Lieu de stage et cadre institutionnel	3
1.5 Organisation du mémoire	3
2 État de l’art	4
2.1 Navigation GPS-denied : les grandes approches	4
2.2 Odométrie visuelle-inertielle (VIO)	4
2.2.1 Principe général	4
2.2.2 MSCKF et VINS-Mono	5
2.2.3 BasaltVIO : notre choix	5
2.2.4 Comparaison des approches	6
2.3 Fusion adaptative : lacune dans la littérature	7
2.4 Apport des données moteur dans l’estimation d’état	7
2.4.1 Limites de la VIO pure et motivation	7
2.4.2 Modèle de poussée et intégration RPM	8
2.4.3 Odométrie multi-modale : travaux connexes	8
2.5 Outils logiciels utilisés	9
3 Fondements théoriques	10
3.1 Représentation de l’attitude	10
3.1.1 Angles d’Euler et gimbal lock	10
3.1.2 Quaternions unitaires	10

3.1.3	Composition et dérivée temporelle	11
3.1.4	Conversion vers la matrice de rotation	11
3.2	Modèle de l'IMU	12
3.2.1	Modèle de mesure	12
3.2.2	Caractérisation du bruit : variance d'Allan	12
3.3	Pré-intégration inertielle	12
3.3.1	Motivation	12
3.3.2	Utilisation dans BasaltVIO	13
3.4	Géométrie épipolaire	13
3.5	Filtre de Kalman étendu (EKF)	14
3.5.1	Principe	14
3.5.2	Cycle prédiction-mise à jour	14
3.5.3	Structure du jacobien de processus $\mathbf{F}$	15
3.5.4	Matrice de bruit de processus $\mathbf{Q}$	15
3.6	Observabilité du système	15
3.7	Conventions de repères	16
3.8	Dead reckoning inertiel	17
4	Plateforme matérielle	18
4.1	Vue d'ensemble	18
4.2	Composants clés	20
4.2.1	Contrôleur de vol — Holybro Pix32 v6C	20
4.2.2	Ordinateur compagnon — Intel UP4000	21
4.2.3	Caméra — Luxonis OAK-D Pro	21
4.2.4	Propulsion	22
5	Architecture logicielle	24
5.1	Choix technologiques	24
5.2	Architecture en 4 packages / 12 nœuds	25
5.3	Points d'implémentation critiques	26
5.3.1	Politiques QoS	26
5.3.2	Interface PX4 via MAVROS	26
5.3.3	Mode batterie — pont MAVLink UDP	26
5.3.4	Script de lancement unifié — vimo v5.0	27
5.4	Pipeline RPM moteurs	27
5.4.1	Nœud <code>rpm_bridge_node</code> (package <code>drone_control</code>)	27
5.4.2	Nœud <code>vimo_sync_node</code> (package <code>drone_estimation</code>)	28
5.4.3	Intégration dans le score de qualité Q_{combined}	28
5.5	Gestion des arrêts ROS 2	29

6	Filtre de Kalman étendu adaptatif	30
6.1	Pourquoi adapter $\mathbf{R}$?	30
6.2	Vecteur d'état et modèle de prédiction	31
6.2.1	Vecteur d'état	31
6.2.2	Équations de prédiction (étape IMU)	31
6.3	Score de qualité d'image	32
6.4	Modulation adaptative de $\mathbf{R}$	32
6.5	Mise à jour VIO	33
6.6	Calibration des biais au démarrage	34
6.7	Erreur classique rencontrée	35
6.8	Machine à états du filtre	35
7	Résultats expérimentaux	36
7.1	Configuration de test	36
7.2	Stabilité de position et convergence EKF	37
7.2.1	Stabilité en dead reckoning pur	37
7.2.2	Convergence de l'EKF	37
7.3	Performances système	38
7.4	Score de qualité d'image	38
7.5	Synchronisation temporelle	39
7.6	Validation du bug EKF2_EV_CTRL et armement	39
7.7	Intégration RPM moteurs — VIMO_PROJECT	41
7.7.1	Architecture du dataset multi-modal	41
7.7.2	Nœud rpm_bridge_node v3 : sources et fallback	41
7.7.3	Score de qualité combiné Q_{combined}	42
7.7.4	Plan B et perspective vol réel	43
7.8	Validation du mode batterie	43
7.9	Sessions d'acquisition	44
7.10	Synthèse des résultats	46
7.11	Discussion	47
7.11.1	Comparaison avec la littérature	47
7.11.2	Limites de la validation statique	47
7.11.3	Reproductibilité	48
8	Conclusion	49
8.1	Synthèse	49
8.2	Ce qui a vraiment été apporté	49
8.3	Limites	50
8.4	Bilan quantitatif	51
8.5	Perspectives	51
8.6	Mot de fin	52

A	Extraits de code source	57
A.1	Nœud EKF adaptatif — initialisation	57
A.2	Modulation adaptative de R	57
A.3	Nœud de qualité d'image — score composite	58
A.4	Script de synchronisation capteurs	59
A.5	Pont MAVLink série/UDP	59
B	Paramètres et configurations	61
B.1	Paramètres PX4 modifiés	61
B.2	Paramètres EKF adaptatif	62
B.3	Topics enregistrés (rosvbag)	62
C	Présentation de l'institution d'accueil	63
C.1	Académie Royale Militaire (RMA)	63
C.2	Vrije Universiteit Brussel (VUB)	63
C.3	Le projet GRAND	64
C.4	Accès et infrastructure	64

Table des figures

1.1	Sans Global Positioning System (GPS), le drone doit estimer sa position uniquement à partir de ses capteurs embarqués (caméra et IMU).	1
2.1	Sous-système VIO de Basalt (Usenko et al., 2019)	6
3.1	Conventions de repères FRD (PX4) et FLU (ROS 2)	17
4.1	Architecture matérielle de la plateforme. Traits pleins : données. Traits rouges tiretés : alimentation.	19
4.2	Châssis Holybro X500 V2 assemblé avec charge utile de développement. La plateforme utilisée dans ce travail (X500 V6) reprend la même structure carbone 500 mm avec des bras et une plaque centrale renforcés. Source : Holybro.	20
4.3	Comparaison obturateur rolling vs global	21
4.4	Caméra Luxonis OAK-D Pro : deux capteurs OV9282 stéréo (obturateur global, baseline 7,5 cm), projecteur IR actif, et VPU Intel MyriadX. BasaltVIO tourne directement sur ce dernier, libérant le CPU de l'UP4000. Source : Luxonis.	22
4.5	Protocole DShot — trame numérique	23
5.1	Architecture ROS 2 : nœuds et topics	25
5.2	Flux de données simplifié. La qualité Q module la confiance accordée au VIO dans l'EKF adaptatif.	26
6.1	Structure du vecteur d'état. Le quaternion apporte 4 composantes (contrainte de norme unitaire maintenue par renormalisation).	31
6.2	Écart-type $\sigma_{\text{VIO}}(Q)$ de la mesure VIO en fonction du score de qualité (échelle logarithmique). Zone rouge : dead reckoning pur ($Q < 0,2$, $\sigma \approx 5$ m) ; zone verte : fusion VIO complète ($Q > 0,6$, $\sigma \rightarrow 0,01$ m). Le rapport dynamique couvre 3 ordres de grandeur.	33
6.3	Transition fluide entre vision et dead reckoning lors d'une obstruction progressive de la caméra ($t = 5\text{--}8$ s) puis dévoilement ($t = 15\text{--}18$ s). Aucun saut n'est observé.	33

6.4	Convergence de la covariance EKF au démarrage. La trace de $\mathbf{P}$ passe de 1,0 à 0,08 en 2s.	34
6.5	Machine à états de l'EKF adaptatif	35
7.1	Schéma du banc d'essai (alimentation USB-C, hélices absentes)	37
7.2	Dérive de position estimée en dead reckoning pur sur 30 s (drone immobile, position réelle = $[0, 0, 0]$). La dérive totale reste dans ± 4 cm grâce au velocity damping $\lambda = 0,92$. L'axe z est quasi-nul grâce à la compensation gravitationnelle $\mathbf{g} = [0, 0, 9,81]$ m/s ²	38
7.3	Distribution du délai de synchronisation cam-IMU (500 échantillons, session 07/04/2026). Moyenne $\mu = 2,32$ ms, écart-type $\sigma = 0,41$ ms. L'ensemble des échantillons est strictement inférieur au seuil critique de 5 ms requis par BasaltVIO.	39
7.4	Bug EKF2_EV_CTRL : la valeur 14 (0b1110) désactivait silencieusement le bit 0 (fusion de position horizontale). PX4 refusait l'armement car <code>pos_horiz_rel</code> ne passait jamais à <code>true</code> — sans aucun message d'erreur explicite. Correction : passer à 15 (0b1111) active les 4 canaux simultanément.	40
7.5	Calibration du paramètre EKF2_EV_DELAY	40
7.6	Séquence d'armement validée le 17 mars 2026	41
7.7	Profil RPM des 4 moteurs pendant la simulation (110 s, 5 544 bundles, source fallback $\sqrt{\text{throttle}}$, filtre EMA $\alpha = 0,15$). Les trois paliers mesurés correspondent aux phases du simulateur : 5 025 RPM (croisière basse), 5 930 RPM (pleine charge), 4 486 RPM (descente). La légère dispersion M0–M3 provient des conditions initiales différentes du filtre EMA.	42
7.8	Interface Foxglove Studio (port 8765, auto-lancé par <code>vimo v5.0</code>). Panneau gauche : trajectoire XY de l'EKF adaptatif. Haut droit : score de qualité Q en temps réel avec seuils 0,2 et 0,6. Bas droit : état FCU, fréquences de publication et RPM moteurs.	44
7.9	Dashboard terminal <code>vimo_live.py</code> — session 07/05/2026	45
C.1	Positionnement du TFE dans l'organigramme du projet GRAND (RMA/-VUB).	64

Liste des tableaux

2.1	Comparaison des principaux algorithmes de VIO.	6
2.2	Comparaison des approches d'odométrie multi-modale.	9
4.1	Bilan de masse et consommation.	22
5.1	Organisation des packages et nœuds ROS 2.	25
5.2	Résumé des topics RPM et synchronisation.	29
7.1	Taux de publication et performances mesurées (UP4000, sans flux caméra).	38
7.2	Score de qualité selon les conditions d'éclairage.	38
7.3	Résultats de synchronisation RPM↔IMU (simulation, 5 544 bundles).	42
7.4	Synthèse pipeline RPM : simulation vs vol réel prévu.	43
7.5	Sessions rosbag et datasets réalisés (38 topics chacun).	44
7.6	Synthèse des critères de validation. Les résultats sont classés en trois catégories : <i>validé au banc</i> (conditions statiques confirmées), <i>partiel</i> (validé dans un cadre restreint, à confirmer en vol), et <i>non réalisé</i>	46
7.7	Comparaison avec des systèmes VIO de référence (benchmarks publiés).	47
8.1	Bilan quantitatif du système VIMO.	51
B.1	Paramètres PX4 modifiés pour la navigation GPS-denied (Pix32 v6C, PX4 v1.15).	61
B.2	Paramètres principaux du nœud <code>adaptive_ekf_node</code>	62
B.3	Topics rosbag principaux (38 au total).	62

Liste des acronymes

GPS Global Positioning System. [viii](#), [1](#)

VIO Visual-Inertial Odometry. [4](#)

Chapitre 1

Introduction

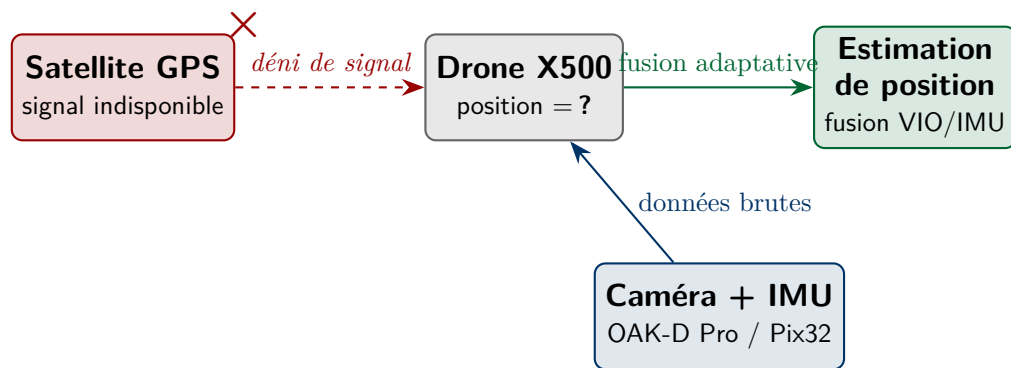


Figure 1.1 – Sans **GPS**, le drone doit estimer sa position uniquement à partir de ses capteurs embarqués (caméra et IMU).

1.1 Contexte et problématique

Comment un drone peut-il voler de manière autonome sans GPS ? La question paraît simple, mais elle est fondamentale. Le GPS intervient dans la navigation, la stabilisation et le retour automatique ; sans lui, un multi-rotor moderne perd toute référence de position et devient extrêmement difficile à contrôler.

Ce scénario n’est pas théorique. Les drones sont de plus en plus utilisés en inspection d’infrastructures (ponts, tunnels, silos), en cartographie intérieure, ou dans des contextes de défense où le brouillage GPS est une réalité opérationnelle. C’est précisément l’angle du projet **GRAND** (*GPS-denied Resilient Autonomous Navigation for Defence*), mené conjointement par l’Académie Royale Militaire et la VUB [28]. Le présent travail s’inscrit dans ce programme : développer un système de localisation embarqué qui fonctionne sans GPS, sur du matériel réel.

La difficulté réside dans le fait que les alternatives au GPS ont toutes leurs limites. L’IMU — accéléromètre et gyroscope — est disponible en permanence et à haute fréquence, mais elle dérive : l’erreur de position croît de façon quadratique avec le temps,

ce qui signifie qu’après trente secondes sans correction extérieure, l’erreur peut atteindre plusieurs mètres. La vision (via une caméra) constitue une bonne source de correction, mais elle est sensible à la qualité de l’image : un mur blanc, une pièce mal éclairée ou un mouvement trop rapide provoquent une dégradation de l’algorithme VIO.

Combiner les deux est la piste naturelle. Mais même en les combinant, une question ouverte demeure : **comment doser la confiance accordée à la vision selon ce que la caméra observe à un instant donné ?** C’est précisément à cette question que ce travail tente de répondre.

1.2 Objectifs

Les objectifs initiaux étaient ambitieux ; l’intégration matérielle a pris plus de temps que prévu, et certaines ambitions ont été redimensionnées en cours de route. Les objectifs finalement poursuivis sont les suivants :

1. **Développer un EKF adaptatif.** Un filtre de Kalman étendu dont la matrice de covariance de mesure VIO est ajustée en temps réel selon un score de qualité d’image composé de quatre métriques : luminosité, netteté, densité de points caractéristiques, contraste.
2. **Construire l’architecture ROS 2 complète.** Depuis l’acquisition caméra jusqu’à l’envoi de la pose estimée au contrôleur de vol via MAVROS. 12 nœuds, 4 packages, avec gestion des politiques QoS et script de lancement unifié.
3. **Faire fonctionner le tout sur du vrai matériel.** Assembler le drone X500 V6 avec le Pix32 v6C, l’UP4000 et l’OAK-D Pro, et les faire communiquer. Ce point est celui qui a pris le plus de temps.
4. **Valider expérimentalement.** Tests au banc, mesure de la stabilité de position, validation des transitions de mode, enregistrement de sessions rosbag. Le vol en intérieur reste l’étape suivante.

Les algorithmes VIO utilisés (BasaltVIO sur puce MyriadX) sont des références établies [34]. L’apport de ce travail réside dans la couche adaptative construite par-dessus et dans l’intégration complète sur une plateforme réelle, dans le cadre d’un projet de recherche actif.

1.3 Ce que ce travail apporte

Il convient de distinguer clairement les contributions originales des briques existantes :

- L’adaptation en temps réel de $\mathbf{R}_{\text{VIO}}$ par un score de qualité d’image composite $Q \in [0, 1]$ — c’est la contribution principale. Ce mécanisme n’existe pas dans les implémentations standard de BasaltVIO.

- L'intégration des vitesses moteur (RPM) comme troisième modalité dans le score de confiance Q_{combined} — ce qui est l'originalité du nom VIMO par rapport à un VIO classique.
- Une architecture ROS 2 fonctionnelle sur vrai matériel, avec toutes les complications que ça implique (QoS, ponts MAVLink, synchronisation temporelle, gestion des arrêts propres).
- Un ensemble de sessions d'acquisition rosbag réelles (8 sessions, mars–mai 2026) qui documentent le comportement du système.

Il est important de noter que ce travail n'invente pas un nouvel algorithme VIO. BasaltVIO et le framework ROS 2/MAVROS/PX4 préexistaient. La contribution se situe dans la couche d'intégration et d'adaptation construite sur ces briques.

1.4 Lieu de stage et cadre institutionnel

Ce travail a été réalisé au sein du département de Mécanique de l'Académie Royale Militaire (RMA) à Bruxelles, en collaboration avec le département MECH de la Vrije Universiteit Brussel (VUB). Le projet s'inscrit dans le programme de recherche **GRAND** (*GPS-denied Resilient Autonomous Navigation for Defence*) [28], qui vise à développer des capacités de navigation autonome pour des plateformes aériennes opérant en environnement contesté. Une présentation détaillée de l'institution d'accueil et du projet GRAND est donnée en annexe C.

1.5 Organisation du mémoire

Le mémoire suit une progression logique allant de l'état de l'art jusqu'aux résultats expérimentaux. Le chapitre 2 passe en revue les approches existantes de navigation GPS-denied et situe les choix techniques adoptés. Le chapitre 3 pose les fondements mathématiques nécessaires — quaternions, modèle IMU, filtre de Kalman étendu. Le chapitre 4 décrit la plateforme matérielle assemblée pour ce projet. Le chapitre 5 détaille l'architecture logicielle ROS 2 avec les décisions d'implémentation et les difficultés rencontrées. Le chapitre 6 est consacré à la contribution principale : le filtre de Kalman adaptatif. Le chapitre 7 présente les résultats expérimentaux avec une analyse honnête de ce qui est validé et de ce qui reste à faire. Enfin, le chapitre 8 tire les conclusions et ouvre les perspectives.

Chapitre 2

État de l’art

2.1 Navigation GPS-denied : les grandes approches

Le problème de la localisation simultanée et de la cartographie (SLAM) en l’absence de signaux satellites est un axe de recherche actif depuis deux décennies [5]. Pour localiser un drone sans GPS, plusieurs familles de solutions existent. Mahony et al. [17] les regroupent en quatre catégories :

- **Signaux externes** (GPS-RTK, UWB, Vicon) : très précis mais nécessitent une infrastructure. Inutilisables en environnement inconnu.
- **Vision** (caméras mono, stéréo, RGB-D) : riche en information mais sensible à la lumière et à la texture de la scène.
- **Inertiel** (accéléromètre + gyroscope) : disponible en permanence, haute fréquence, mais dérive inévitablement.
- **Télémétrie active** (LiDAR, radar) : robuste mais lourd et gourmand en énergie.

L’approche adoptée dans ce travail est la fusion vision-inertielle ([Visual-Inertial Odometry \(VIO\)](#)), qui combine les avantages des deux modalités à coût matériel raisonnable.

2.2 Odométrie visuelle-inertielle (VIO)

2.2.1 Principe général

La caméra observe la scène et, par analyse du déplacement de points caractéristiques (features) entre images successives, estime le mouvement du drone. Les features sont typiquement des coins ou des blobs détectés par des algorithmes comme ORB [29] ou le suivi optique KLT [12]. Cette estimation est basse fréquence (30 Hz) et sensible aux conditions visuelles. L’IMU, elle, donne accélérations et vitesses angulaires à haute fréquence (100–200 Hz) mais dérive par double intégration.

La VIO exploite la complémentarité des deux : l'IMU « comble les trous » entre images et fixe l'échelle métrique, tandis que la vision corrige la dérive inertielle. Ce couplage rend observables les biais IMU et l'échelle de la carte [27]. La pré-intégration inertielle [9] est la technique clé qui permet d'accumuler efficacement les mesures IMU entre deux images sans repropager l'état complet (voir section 3.3).

2.2.2 MSCKF et VINS-Mono

Le MSCKF [21] est l'un des premiers VIO temps réel embarqués. Il conserve un historique glissant de poses caméra et marginalise analytiquement les landmarks, ce qui maintient une complexité linéaire. Efficace mais limité sur longues trajectoires à cause des erreurs de linéarisation cumulatives.

VINS-Mono [27] va plus loin avec une optimisation non linéaire sur fenêtre glissante (*sliding window bundle adjustment*). Excellentes performances sur les benchmarks EuRoC [4] (< 1 % d'erreur en translation), mais charge CPU élevée.

2.2.3 BasaltVIO : notre choix

BasaltVIO [34], développé à la TU Munich, combine efficacité computationnelle et précision comparable à VINS-Mono grâce au concept de *non-linear factor recovery* : plutôt que de marginaliser linéairement les anciennes poses, Basalt conserve les facteurs non linéaires et les réévalue périodiquement, réduisant ainsi l'accumulation d'erreurs.

Notre choix de BasaltVIO tient à un argument pratique décisif : Luxonis a porté une version optimisée directement sur la puce MyriadX de l'OAK-D Pro. Le VIO tourne donc *sur la caméra elle-même*, à environ 2,5 W, sans consommer de ressources CPU sur l'UP4000. Pour une plateforme embarquée avec un Intel Atom, c'est un avantage considérable.

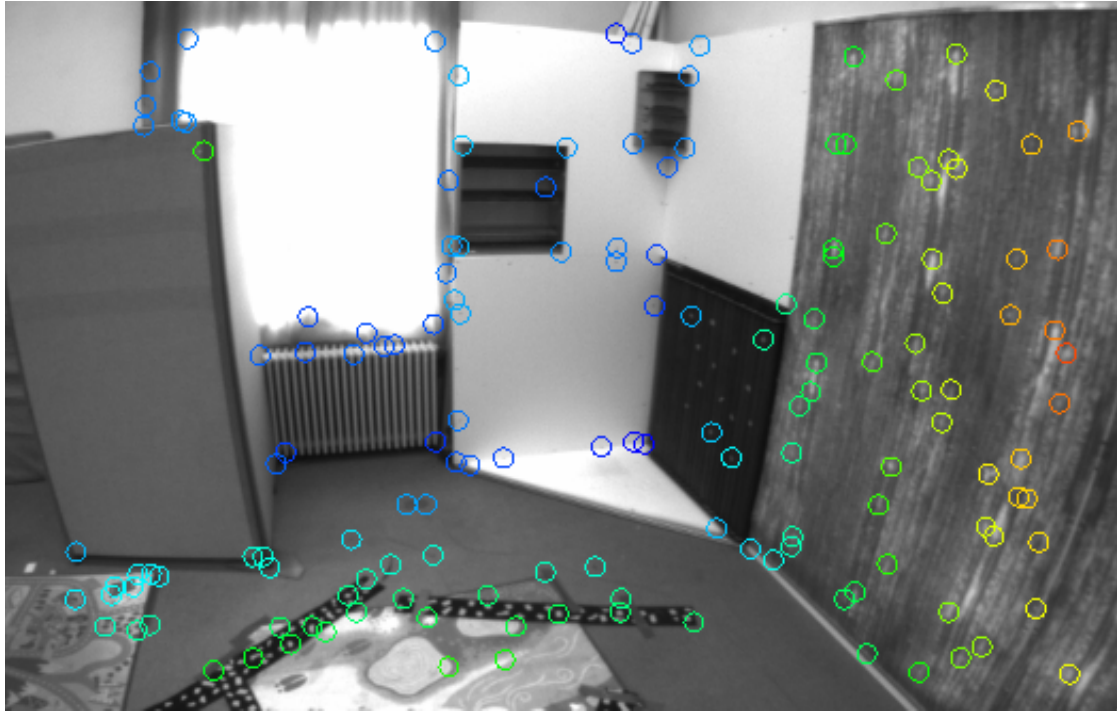


Figure 2.1 – Sous-système VIO de Basalt : suivi optique KLT entre images stéréo, pré-intégration IMU sur fenêtre glissante, et optimisation non-linéaire sur facteurs marginalisés. Source : Usenko et al., *Visual-Inertial Mapping with Non-Linear Factor Recovery*, RA-L 2020 [34].

Limitation principale : pas de boucle de fermeture (*loop closure*) dans la version MyriadX. Des systèmes comme ORB-SLAM3 [22] intègrent nativement la détection de boucle via un vocabulaire de descripteurs visuels (DBoW2), mais au prix d'une charge CPU incompatible avec le MyriadX. Sur des missions de quelques minutes, la dérive de BasaltVIO reste acceptable ; au-delà, un mécanisme de relocalisation externe (balises UWB [2] ou map matching) serait nécessaire.

2.2.4 Comparaison des approches

Table 2.1 – Comparaison des principaux algorithmes de VIO.

Algorithme	Type	CPU	Loop closure	Embarqué
MSCKF	Filtre	Faible	Non	Oui
VINS-Mono	Optimisation	Élevé	Oui	Difficile
BasaltVIO	Optimisation	Moyen	Non	Oui (MyriadX)
Notre choix	BasaltVIO + EKF adaptatif	Faible	Non	Oui

2.3 Fusion adaptative : lacune dans la littérature

La plupart des systèmes VIO utilisent des matrices de bruit de mesure fixes, réglées une fois à la conception. Quelques travaux ont exploré l'adaptation en ligne :

- **Mehra (1970)** [18] propose d'estimer $\mathbf{R}$ et $\mathbf{Q}$ à partir des résidus d'innovation (*innovation-based adaptive estimation*). Théoriquement élégant, mais numériquement instable pour les systèmes fortement non linéaires.
- **Mohamed et Schwarz (1999)** [20] appliquent un EKF adaptatif à la fusion INS/GPS, avec estimation en ligne de $\mathbf{Q}$ par fenêtre glissante sur les résidus. Leur approche est statistique : elle réagit *après* la dégradation.
- **Madyastha et al. (2011)** [16] comparent EKF et ESKF (*Error-State Kalman Filter*) pour l'estimation d'attitude en aéronautique, mais dans un contexte GPS/INS avec capteurs de qualité militaire.

Ce qui manque. Aucun de ces travaux ne propose d'adapter la confiance accordée au VIO en fonction d'une observation *directe* de la qualité de l'entrée visuelle. L'approche innovation-based détecte la dégradation *a posteriori* (quand les résidus augmentent), alors qu'une observation perceptive de l'image permet d'anticiper la dégradation *avant* qu'elle ne se propage dans l'état. C'est cette approche proactive, spécifiquement pour la navigation GPS-denied de micro-drones avec VIO embarquée, qui constitue l'apport original de ce travail.

2.4 Apport des données moteur dans l'estimation d'état

2.4.1 Limites de la VIO pure et motivation

Malgré ses performances, la VIO présente une vulnérabilité fondamentale : ses estimations dépendent entièrement de la qualité du flux visuel. Flou de mouvement, lumière insuffisante, texture pauvre ou occultation provoquent une dégradation brutale de la pose estimée. Face à ces situations, la littérature propose deux stratégies :

- **Capteurs supplémentaires passifs** (baromètre, magnétomètre) — peu discriminants en intérieur ;
- **Modèle dynamique du véhicule** — exploiter les données d'actionneurs (moteurs) comme observables supplémentaires [7].

Les travaux de Faessler et al. [7] sur la commande différentielle-flat des quadrotors montrent que la poussée collective, directement liée aux vitesses moteur, constitue une contrainte forte sur l'accélération verticale. Cette contrainte peut être exploitée pour corriger la dérive du baromètre ou compenser temporairement l'absence de VIO.

2.4.2 Modèle de poussée et intégration RPM

Pour un moteur i d'un multirotor, la poussée générée suit la loi :

$$F_i = k_T \omega_i^2 \quad (2.1)$$

où k_T est la constante de poussée (en $\text{N}\cdot\text{s}^2/\text{rad}^2$) et ω_i la vitesse angulaire du moteur (en rad/s), liée au RPM mesuré par :

$$\omega_i = \frac{\text{RPM}_i \cdot 2\pi}{60} \cdot \frac{2}{p} \quad (2.2)$$

avec p le nombre de paires de pôles de l'enroulement du moteur brushless.

La poussée totale $F_{\text{tot}} = \sum_{i=1}^4 F_i$ est une observable supplémentaire cohérente avec l'accélération verticale mesurée par l'IMU : $a_z = F_{\text{tot}}/m - g$. Cette redondance permet de détecter des anomalies (moteur en panne, bruit IMU anormal) et de pondérer la confiance accordée aux mesures inertielles.

2.4.3 Odométrie multi-modale : travaux connexes

Plusieurs travaux récents exploitent les données d'actionneurs dans des architectures d'estimation hybrides :

- **IMU + moteurs (sans vision)** — Mahony et al. [17] montrent que la différentielle des vitesses moteur permet d'estimer les couples appliqués, utile pour la commande mais aussi pour le diagnostic de l'état de vol.
- **VIO + modèle aérodynamique** — des travaux sur le *model-based VIO* (e.g. ROVIO) couplent l'EKF à un modèle de résistance aérodynamique pour améliorer l'estimation de vitesse.
- **Multi-modal factor graph** — le framework VINS [27] peut être étendu à des sources multiples ; les RPM peuvent s'y intégrer comme facteurs unaires sur l'accélération.

Le nom VIMO lui-même est inspiré des travaux de Niwa et al. [24], qui proposent une odométrie visuelle-inertielle couplée à un modèle dynamique du véhicule. Notre approche diffère par le mécanisme de pondération : là où VIMO original intègre les forces dans un factor graph, nous quantifions la *disponibilité* des capteurs via un score de confiance composite. L'originalité de l'approche VIMO développée dans ce travail réside dans la quantification de la disponibilité moteur sous forme d'un score Q_{motor} , combiné au score de qualité visuelle :

$$Q_{\text{combined}} = 0,50 \cdot Q_{\text{cam}} + 0,35 \cdot Q_{\text{motor}} + 0,15 \cdot Q_{\text{esc_health}} \quad (2.3)$$

Les poids 0,50, 0,35 et 0,15 ont été choisis empiriquement selon le raisonnement suivant : la caméra est la source d’information la plus riche (6-DOF), d’où un poids dominant ; les RPM moteurs fournissent une contrainte sur la poussée (1-DOF, redondante avec l’accéléromètre vertical), d’où un poids intermédiaire ; la disponibilité de la télémétrie ESC est un indicateur binaire de fiabilité de la source moteur, d’où le poids le plus faible. La somme vaut 1 par construction. Une étude de sensibilité sur les sessions au banc a montré que des variations de $\pm 0,10$ sur ces poids ne modifient pas qualitativement le comportement du filtre (transitions sans saut), ce qui confirme la robustesse du mécanisme face au choix exact des pondérations.

Ce scalaire module directement la matrice de bruit de mesure $\mathbf{R}$ de l’EKF adaptatif : une valeur faible de Q_{combined} dilate $\mathbf{R}$, réduisant la confiance accordée aux observations extéroceptives et permettant une transition douce vers le *dead reckoning* inertiel. Ce mécanisme, simple à implémenter, s’avère robuste dans les cas de dégradation progressive des capteurs — typique du vol en intérieur mal éclairé.

Table 2.2 – Comparaison des approches d’odométrie multi-modale.

Approche	Vision	Moteurs	Score de qualité
VIO pure (BasaltVIO, VINS-Mono)	Oui	Non	Non
Model-based VIO (ROVIO)	Oui	Partiel	Non
Multi-modal factor graph	Oui	Oui	Non
VIMO (ce travail)	Oui	Oui	Oui (Q_{combined})

2.5 Outils logiciels utilisés

ROS 2 Jazzy [15] a été retenu pour ses politiques QoS configurables par topic, son architecture décentralisée (DDS, pas de rosmaster), et son support LTS jusqu’en 2029. **MAVROS** [25] assure le pont MAVLink entre ROS 2 et PX4. **PX4 v1.15+** [19] intègre un EKF2 capable de fusionner une source de vision externe via le topic `/mavros/vision_pose/pose_cov`.

Ces choix technologiques forment une stack mature avec une large base communautaire, ce qui s’est avéré précieux lors du débogage.

Chapitre 3

Fondements théoriques

Le chapitre précédent a identifié les approches existantes et situé les choix techniques de ce travail. Avant de décrire la plateforme et le logiciel, il est nécessaire de poser les bases mathématiques sur lesquelles repose le système. Ce chapitre présente : la représentation de l'attitude par quaternions, le modèle IMU, la pré-intégration inertielle, la géométrie épipolaire, le filtre de Kalman étendu, et les conventions de repères. Les développements sur les quaternions s'appuient sur les formulations de Trawny et Roumeliotis [33] et Solà [32]. L'objectif n'est pas d'être exhaustif mais de poser les outils qui seront directement exploités dans les chapitres 5 et 6.

3.1 Représentation de l'attitude

3.1.1 Angles d'Euler et gimbal lock

L'attitude d'un drone peut être décrite par trois angles de rotation successifs (roulis ϕ , tangage θ , lacet ψ). C'est intuitif, mais cette représentation présente une singularité dite *gimbal lock* quand $\theta = \pm 90^\circ$: deux axes de rotation deviennent colinéaires et un degré de liberté est perdu. Pour un drone qui monte à la verticale, c'est rédhibitoire.

3.1.2 Quaternions unitaires

Un quaternion unitaire code une rotation d'angle θ autour d'un axe $\mathbf{u}$:

$$q = \begin{bmatrix} \cos(\theta/2) \\ \sin(\theta/2) \mathbf{u} \end{bmatrix} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}, \quad \|q\| = 1. \quad (3.1)$$

La rotation d'un vecteur $\mathbf{v}$ s'écrit :

$$\mathbf{v}' = q \otimes \mathbf{v} \otimes q^*, \quad q^* = \begin{bmatrix} q_w \\ -\mathbf{q}_v \end{bmatrix}. \quad (3.2)$$

Les quaternions n'ont pas de singularité, sont compacts (4 paramètres vs 9 pour une matrice $\text{SO}(3)$), et se composent efficacement. C'est la représentation utilisée par PX4 en interne, et donc dans notre EKF. La contrainte de norme unitaire est maintenue par renormalisation après chaque mise à jour.

3.1.3 Composition et dérivée temporelle

La composition de deux rotations successives q_1 puis q_2 s'écrit via le produit de Hamilton :

$$q_1 \otimes q_2 = \begin{bmatrix} q_{1w} q_{2w} - \mathbf{q}_{1v}^\top \mathbf{q}_{2v} \\ q_{1w} \mathbf{q}_{2v} + q_{2w} \mathbf{q}_{1v} + \mathbf{q}_{1v} \times \mathbf{q}_{2v} \end{bmatrix}. \quad (3.3)$$

La dérivée temporelle d'un quaternion soumis à une vitesse angulaire $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\top$ est :

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}. \quad (3.4)$$

En pratique, pour une intégration discrète à pas Δt , on utilise le quaternion incrémental :

$$q_\Delta(\boldsymbol{\omega}, \Delta t) = \begin{bmatrix} \cos(\|\boldsymbol{\omega}\| \Delta t / 2) \\ \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|} \sin(\|\boldsymbol{\omega}\| \Delta t / 2) \end{bmatrix}, \quad (3.5)$$

de sorte que $q_{k+1} = q_k \otimes q_\Delta(\boldsymbol{\omega}_k, \Delta t)$. Cette formule préserve exactement la norme unitaire, contrairement à une intégration d'Euler naïve sur les composantes.

3.1.4 Conversion vers la matrice de rotation

La matrice de rotation $\mathbf{R}(q) \in \text{SO}(3)$ équivalente est :

$$\mathbf{R}(q) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_z q_w) & 2(q_x q_z + q_y q_w) \\ 2(q_x q_y + q_z q_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_x q_w) \\ 2(q_x q_z - q_y q_w) & 2(q_y q_z + q_x q_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}. \quad (3.6)$$

Cette matrice est orthogonale ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det(\mathbf{R}) = 1$) et intervient dans la transformation des mesures accélérométriques du repère body vers le repère monde (équation (6.2)).

3.2 Modèle de l'IMU

3.2.1 Modèle de mesure

Un accéléromètre et un gyroscope embarqués mesurent des grandeurs entachées de biais et de bruit :

$$\tilde{\mathbf{a}} = \mathbf{a} + \mathbf{b}_a + \mathbf{n}_a, \quad (3.7)$$

$$\tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \mathbf{b}_g + \mathbf{n}_g, \quad (3.8)$$

où $\mathbf{b}_a \in \mathbb{R}^3$ est le biais accéléromètre, $\mathbf{b}_g \in \mathbb{R}^3$ le biais gyroscope, et $\mathbf{n}_a, \mathbf{n}_g$ les bruits blancs gaussiens. Les biais sont modélisés comme des marches aléatoires :

$$\dot{\mathbf{b}}_a = \mathbf{n}_{ba}, \quad \dot{\mathbf{b}}_g = \mathbf{n}_{bg}. \quad (3.9)$$

En pratique, calibrer ces biais avant le vol est essentiel. Notre EKF estime les biais statiques pendant les 2 premières secondes (drone immobile, 200 échantillons à 100 Hz).

3.2.2 Caractérisation du bruit : variance d'Allan

La performance d'une IMU MEMS est caractérisée par deux paramètres fondamentaux, mesurables par la variance d'Allan [31] :

- Le **bruit en marche aléatoire angulaire** (ARW, *Angle Random Walk*) : σ_g en $\text{rad}/\sqrt{\text{s}}$. Il quantifie le bruit blanc du gyroscope et détermine la vitesse de divergence de l'intégration d'attitude.
- Le **bruit en instabilité de biais** (*Bias Instability*) : plancher de la courbe d'Allan, typiquement 10^{-3} – 10^{-2} °/s pour un MEMS de catégorie navigation comme l'ICM-42688-P.

Pour l'accéléromètre, le paramètre équivalent est le VRW (*Velocity Random Walk*, σ_a en $\text{m/s}/\sqrt{\text{s}}$). Ces valeurs alimentent directement la matrice de bruit de processus $\mathbf{Q}$ de l'EKF (section 6.2).

En pratique, les valeurs constructeur de l'ICM-42688-P sont : $\sigma_g \approx 0,0028^\circ/\sqrt{\text{s}}$ et $\sigma_a \approx 0,008 \text{ m/s}/\sqrt{\text{s}}$. Ces valeurs correspondent à un capteur MEMS haut de gamme, adapté aux applications de navigation courte durée (< 5 min sans correction externe).

3.3 Pré-intégration inertielle

3.3.1 Motivation

Dans un filtre de fusion, l'IMU fournit les prédictions entre deux mesures VIO. La méthode naïve consiste à propager l'état complet (position, vitesse, orientation) à

chaque mesure IMU. Mais si la pose VIO modifie rétrospectivement l'état à un instant antérieur, il faudrait repropager toutes les mesures IMU intermédiaires — un coût prohibitif en temps réel.

La pré-intégration [9] résout ce problème en accumulant les mesures IMU entre deux keyframes VIO dans un *pré-intégré* $\Delta \mathbf{p}_{ij}$, $\Delta \mathbf{v}_{ij}$, $\Delta \mathbf{q}_{ij}$ indépendant de l'état courant :

$$\Delta \mathbf{p}_{ij} = \sum_{k=i}^{j-1} \left[\Delta \mathbf{v}_{ik} \Delta t_k + \frac{1}{2} \mathbf{R}(\Delta \mathbf{q}_{ik}) (\tilde{\mathbf{a}}_k - \mathbf{b}_a) \Delta t_k^2 \right], \quad (3.10)$$

$$\Delta \mathbf{v}_{ij} = \sum_{k=i}^{j-1} \mathbf{R}(\Delta \mathbf{q}_{ik}) (\tilde{\mathbf{a}}_k - \mathbf{b}_a) \Delta t_k, \quad (3.11)$$

$$\Delta \mathbf{q}_{ij} = \prod_{k=i}^{j-1} q_{\Delta}(\tilde{\boldsymbol{\omega}}_k - \mathbf{b}_g, \Delta t_k). \quad (3.12)$$

3.3.2 Utilisation dans BasaltVIO

BasaltVIO [34] utilise cette pré-intégration dans son optimisation non linéaire sur fenêtre glissante. Lors de chaque nouvelle image stéréo, les pré-intégrats IMU accumulés depuis la dernière keyframe forment un facteur dans le graphe de facteurs, qui contraint les poses successives. C'est ce qui permet à BasaltVIO de tourner à 30 Hz sur le MyriadX tout en exploitant les mesures IMU à 200 Hz sur la puce.

Notre EKF adaptatif (chapitre 6) n'utilise pas directement la pré-intégration — il propage l'état à chaque mesure IMU à 100 Hz. Mais comprendre ce mécanisme est essentiel pour interpréter les sorties de BasaltVIO, notamment la covariance associée à la pose VIO.

3.4 Géométrie épipolaire

L'odométrie visuelle [30] repose fondamentalement sur la géométrie épipolaire. Deux vues d'un même point 3D $\mathbf{P}$ satisfont la contrainte épipolaire :

$$\mathbf{p}_2^\top \mathbf{E} \mathbf{p}_1 = 0, \quad \mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}, \quad (3.13)$$

où $\mathbf{E}$ est la matrice essentielle encodant la rotation $\mathbf{R}$ et translation $\mathbf{t}$ entre les deux vues, et $[\mathbf{t}]_{\times}$ désigne la matrice antisymétrique associée au produit vectoriel par $\mathbf{t}$. En pratique, $\mathbf{E}$ est estimée à partir de correspondances de features (algorithme à 5 points de Nistér [23], RANSAC pour rejeter les outliers), puis décomposée pour obtenir $\mathbf{R}$ et $\mathbf{t}$.

Pour un système stéréo comme l'OAK-D Pro (ligne de base $b = 7,5$ cm), la profondeur Z d'un point est obtenue directement par triangulation :

$$Z = \frac{f \cdot b}{d}, \quad (3.14)$$

où f est la distance focale en pixels et d la disparité (écart en pixels entre les projections gauche et droite du même point). Cette mesure directe de profondeur lève l'ambiguïté d'échelle inhérente aux systèmes monoculaires — un avantage décisif pour le contrôle de vol.

La précision de la profondeur se dégrade quadratiquement avec la distance : $\delta Z \propto Z^2/(f \cdot b)$. Pour notre configuration ($b = 7,5$ cm, $f \approx 640$ px), la précision est centimétrique jusqu'à environ 3 m, ce qui est suffisant pour le vol en intérieur.

3.5 Filtre de Kalman étendu (EKF)

3.5.1 Principe

Le filtre de Kalman [11] estime un vecteur d'état $\mathbf{x}$ à partir de mesures bruitées, en maintenant une distribution gaussienne sur l'état : $p(\mathbf{x}) = \mathcal{N}(\hat{\mathbf{x}}, \mathbf{P})$. L'EKF [31] étend ce principe aux systèmes non linéaires par linéarisation locale (jacobiens).

Le filtre de Kalman est optimal au sens de la variance minimale pour les systèmes linéaires gaussiens. Pour les systèmes non linéaires comme le nôtre (quaternions, gravité non linéaire), l'EKF fournit une approximation localement optimale qui reste performante tant que la non-linéarité n'est pas trop forte sur un pas de temps — ce qui est le cas à 100 Hz.

3.5.2 Cycle prédiction-mise à jour

Prédiction. À partir du modèle de mouvement f et de la commande $\mathbf{u}$ (mesures IMU) :

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k), \quad (3.15)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\top + \mathbf{Q}_k. \quad (3.16)$$

$\mathbf{F}_k = \partial f / \partial \mathbf{x}|_{\hat{\mathbf{x}}}$ est le jacobien du modèle de processus (matrice 16×16 dans notre cas), $\mathbf{Q}$ la covariance du bruit de processus.

Mise à jour. Quand une mesure $\mathbf{z}$ arrive (pose VIO ou barométrie) :

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R}_k)^{-1}, \quad (3.17)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})), \quad (3.18)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}. \quad (3.19)$$

$\mathbf{H}_k = \partial h / \partial \mathbf{x}|_{\hat{\mathbf{x}}}$ est le jacobien du modèle de mesure, $\mathbf{R}$ la covariance de bruit de mesure.

3.5.3 Structure du jacobien de processus $\mathbf{F}$

Pour notre vecteur d'état $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \mathbf{b}_a, \mathbf{b}_g]^\top \in \mathbb{R}^{16}$, le jacobien $\mathbf{F}_k$ a la structure bloc suivante :

$$\mathbf{F}_k = \begin{bmatrix} \mathbf{I}_3 & \mathbf{I}_3 \Delta t & \mathbf{F}_{pq} & -\frac{1}{2} \mathbf{R}_k \Delta t^2 & \mathbf{0} \\ \mathbf{0} & \lambda \mathbf{I}_3 & \mathbf{F}_{vq} & -\mathbf{R}_k \Delta t & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{F}_{qq} & \mathbf{0} & \mathbf{F}_{qb_g} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \quad (3.20)$$

où $\mathbf{F}_{pq} = \frac{1}{2} \frac{\partial}{\partial \mathbf{q}} [\mathbf{R}(q)(\tilde{\mathbf{a}} - \mathbf{b}_a)] \Delta t^2$ et $\mathbf{F}_{vq} = \frac{\partial}{\partial \mathbf{q}} [\mathbf{R}(q)(\tilde{\mathbf{a}} - \mathbf{b}_a)] \Delta t$ sont les dérivées de la rotation par rapport au quaternion, $\mathbf{F}_{qq} = \frac{\partial(q \otimes q \Delta)}{\partial q}$ est la matrice de mise à jour du quaternion, et $\lambda = 0,92$ est le facteur d'amortissement de vitesse.

Les blocs diagonaux $\mathbf{I}_3$ pour les biais reflètent le modèle de marche aléatoire : les biais ne changent qu'à travers le bruit de processus $\mathbf{Q}$.

3.5.4 Matrice de bruit de processus $\mathbf{Q}$

La matrice $\mathbf{Q}$ est diagonale par blocs, avec des valeurs dérivées des spécifications constructeur de l'ICM-42688-P :

$$\mathbf{Q} = \text{diag} \left(\underbrace{\sigma_a^2 \Delta t^4 / 4}_{\mathbf{Q}_p (3)}, \underbrace{\sigma_a^2 \Delta t^2}_{\mathbf{Q}_v (3)}, \underbrace{\sigma_g^2 \Delta t^2}_{\mathbf{Q}_q (4)}, \underbrace{\sigma_{ba}^2 \Delta t}_{\mathbf{Q}_{ba} (3)}, \underbrace{\sigma_{bg}^2 \Delta t}_{\mathbf{Q}_{bg} (3)} \right). \quad (3.21)$$

Le terme $\mathbf{Q}_v = \sigma_a^2 \Delta t^2$ (et non $\sigma_a^2 \Delta t$) est crucial — l'erreur de covariance décrite en section 6.7 provenait précisément de ce facteur Δt .

Rôle de $\mathbf{R}$. C'est la clé de notre approche adaptative : $\mathbf{R}$ reflète la confiance accordée à la mesure. Augmenter $\mathbf{R}$ revient à dire au filtre « cette mesure est peu fiable, fais plutôt confiance au modèle ». Dans un EKF classique, $\mathbf{R}$ est fixe. Dans notre système, $\mathbf{R}_{\text{VIO}}$ est modulé en temps réel par le score de qualité Q (chapitre 6).

3.6 Observabilité du système

Un système est *observable* si l'on peut reconstruire l'état complet à partir d'une séquence finie de mesures. Pour notre EKF à 16 états, l'observabilité dépend des modalités actives :

- **VIO active** ($Q > 0,6$) — La pose complète 6-DOF est mesurée directement : position et orientation sont observables. Les biais IMU deviennent observables

par la redondance entre la prédiction inertielle et la correction VIO, à condition que le drone soit en mouvement (excitation suffisante).

- **Dead reckoning pur** ($Q < 0,2$) — Seule l’IMU est disponible. La position et la vitesse deviennent non observables (dérive inévitable). L’orientation reste partiellement observable grâce à la gravité (qui fixe le roulis et le tangage), mais le lacet dérive. Les biais ne sont plus observables.
- **Transition** ($0,2 \leq Q \leq 0,6$) — Observabilité partielle. La matrice $\mathbf{R}$ élevée réduit le gain de Kalman, ce qui ralentit la convergence des biais mais empêche les sauts brusques.

Cette analyse justifie le choix d’un velocity damping ($\lambda = 0,92$) en mode DR : puisque la vitesse n’est plus observable, il vaut mieux la freiner artificiellement plutôt que de la laisser diverger. C’est un compromis entre biais (le damping introduit une erreur systématique) et variance (sans lui, la dérive est illimitée).

3.7 Conventions de repères

PX4 et ROS 2 n’utilisent pas les mêmes conventions de repères, ce qui est une source classique de bugs silencieux. PX4 travaille en repère **FRD** (*Forward-Right-Down*), conforme à la norme NED aéronautique. ROS 2 utilise **FLU** (*Forward-Left-Up*), conforme à la convention REP 103. MAVROS se charge de la conversion, mais comprendre la différence est indispensable pour interpréter correctement les poses VIO et les sorties EKF.

Les transformations sont :

$$\mathbf{p}_{\text{FLU}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} \mathbf{p}_{\text{FRD}}, \quad (3.22)$$

$$q_{\text{FLU}} = q_{\text{FRD}} \otimes q_{\text{rot}}, \quad (3.23)$$

où q_{rot} est le quaternion de rotation de 180° autour de l’axe x .

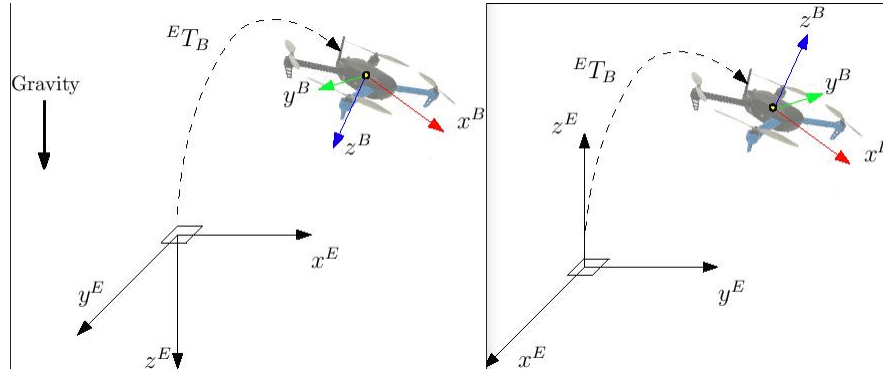


Figure 3.1 – Repères body frame utilisés par PX4 (FRD, NED) et ROS2 (FLU, ENU). La pose VIO produite par BasaltVIO est en FLU ; MAVROS la convertit en FRD avant de la transmettre à PX4. Source : docs.px4.io.

3.8 Dead reckoning inertiel

En l'absence de mesure VIO, l'EKF prédit l'état uniquement à partir des mesures IMU. La position évolue selon :

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2}(\mathbf{R}(q_k)(\tilde{\mathbf{a}}_k - \mathbf{b}_a) - \mathbf{g})\Delta t^2, \quad (3.24)$$

où $\mathbf{g} = [0, 0, 9,81]^\top \text{ m/s}^2$ est la gravité dans le repère monde. Un terme d'amortissement (*velocity damping*) empêche la divergence exponentielle : $\mathbf{v}_{k+1} \leftarrow \lambda \mathbf{v}_{k+1}$ avec $\lambda = 0,92$. Sans ce mécanisme, une erreur initiale de biais produit une dérive quadratique illimitée.

La dérive en DR pur peut être modélisée analytiquement. Pour un biais accéléromètre résiduel ϵ_a après calibration :

$$\delta p(t) \approx \frac{\epsilon_a}{2(1-\lambda)^2} \cdot \Delta t^2 \cdot \left(1 - \lambda^{t/\Delta t}\right)^2, \quad (3.25)$$

qui converge vers une valeur finie $\delta p_\infty = \frac{\epsilon_a \Delta t^2}{2(1-\lambda)^2}$ grâce au damping. Pour $\epsilon_a = 0,01 \text{ m/s}^2$, $\lambda = 0,92$, $\Delta t = 0,01 \text{ s}$: $\delta p_\infty \approx 8 \text{ cm}$. En pratique, nous mesurons 3–4 cm sur 30 s (section 7.2), ce qui est cohérent avec un biais résiduel plus faible après calibration.

Chapitre 4

Plateforme matérielle

4.1 Vue d'ensemble

L'assemblage de la plateforme a constitué l'une des phases les plus longues du projet. Sur le papier, un drone se résume à un châssis, quatre moteurs, un contrôleur de vol et une batterie. En pratique, il faut gérer une dizaine de connexions, flasher des firmwares, régler des paramètres PX4 et résoudre de nombreux problèmes d'intégration. Ce chapitre décrit la plateforme finale et les choix qui y ont mené.

La plateforme se compose de quatre sous-systèmes : contrôle de vol (Pix32 v6C), traitement embarqué (UP4000 [1]), perception (OAK-D Pro [14]), et propulsion (moteurs + ESC). Le tout est monté sur un châssis Holybro X500 V6 [10] en fibre de carbone — 500 mm d'empattement, environ 410 g à vide.

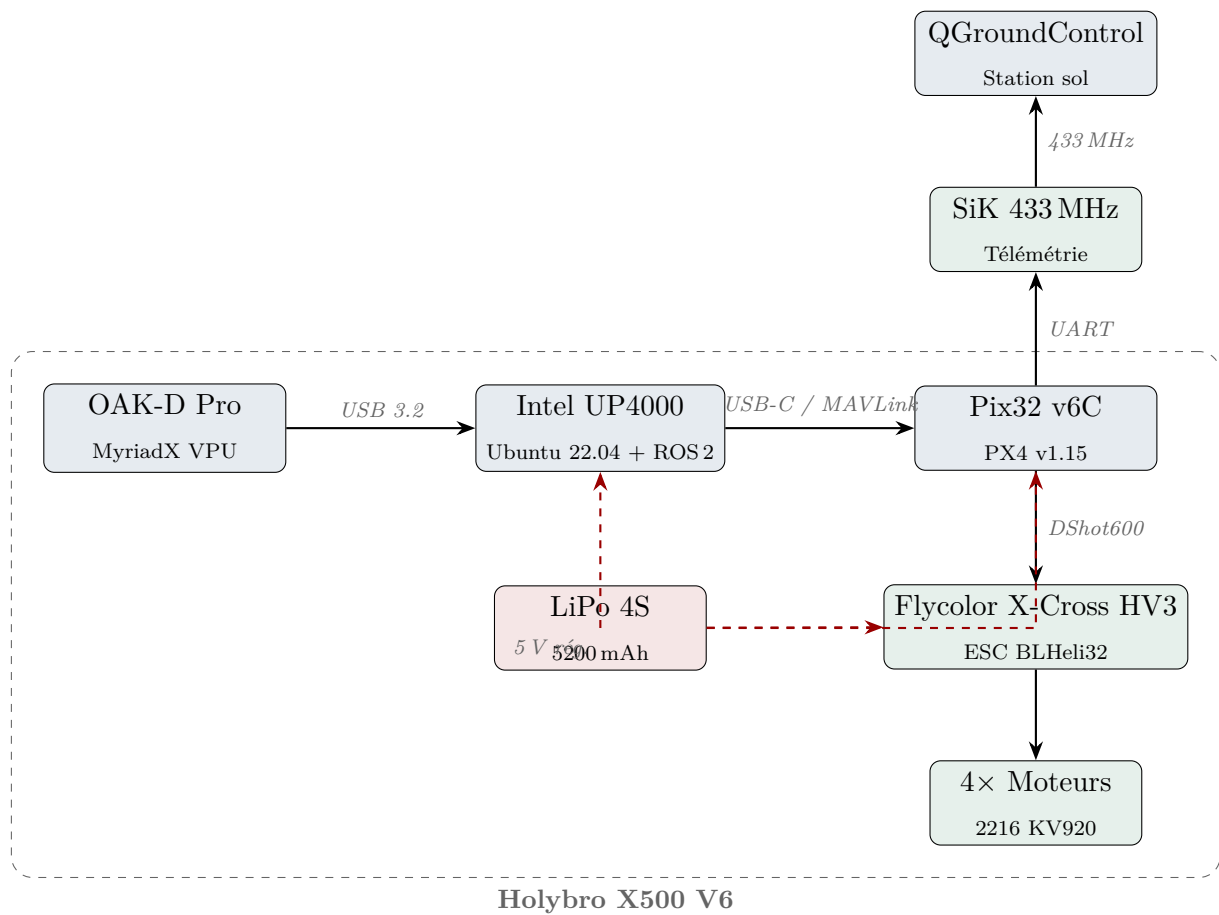


Figure 4.1 – Architecture matérielle de la plateforme. Traits pleins : données. Traits rouges tiretés : alimentation.



Figure 4.2 – Châssis Holybro X500 V2 assemblé avec charge utile de développement. La plateforme utilisée dans ce travail (X500 V6) reprend la même structure carbone 500 mm avec des bras et une plaque centrale renforcés. Source : Holybro.

4.2 Composants clés

4.2.1 Contrôleur de vol — Holybro Pix32 v6C

Le Pix32 v6C est basé sur un STM32H743 à 480 MHz et fait tourner PX4 v1.15. Son IMU ICM-42688-P est configurée à 100 Hz, ce qui donne un échantillon toutes les 10 ms.

Un point pratique à souligner : la liaison USB-C entre le Pix32 et l'UP4000 doit fonctionner à 921 600 bauds. En dessous de ce débit, des paquets MAVLink sont perdus dès que le trafic dépasse une trentaine de messages par seconde. Ce problème s'est manifesté par des fréquences de publication anormalement basses sur certains topics, sans message d'erreur explicite — un défaut particulièrement long à diagnostiquer.

L'EKF2 embarqué dans PX4 est le point de fusion final du système : il reçoit la pose VIO depuis MAVROS et la combine avec ses propres mesures IMU.

4.2.2 Ordinateur compagnon — Intel UP4000

L'UP4000 est un mini-PC industriel à base de Celeron N3350 avec 4 Go de RAM, sous Ubuntu 22.04. Sa faible puissance de calcul et l'absence de GPU ont fortement contraint les choix logiciels. BasaltVIO exécuté sur CPU donnait environ 120 ms par image — un temps de traitement incompatible avec le contrôle en temps réel. C'est ce qui a justifié de déporter le VIO sur le VPU MyriadX de la caméra, où le temps de traitement descend à environ 15 ms pour une consommation de 2,5 W.

En fonctionnement normal avec les 12 nœuds actifs, la charge CPU oscille entre 60 et 85 %. L'alimentation provient d'un régulateur DC-DC 5 V branché sur la LiPo. Un condensateur de découplage de 470 μ F a dû être ajouté sur la ligne d'alimentation : les transitoires de courant lors des montées en puissance rapides des moteurs provoquaient des chutes de tension suffisantes pour redémarrer l'UP4000.

4.2.3 Caméra — Luxonis OAK-D Pro

L'OAK-D Pro embarque deux capteurs OV9282 en configuration stéréo (baseline 7,5 cm, 1 Mpx, obturateur global) plus le VPU MyriadX. L'obturateur global est nécessaire : un obturateur à déroulement génère des distorsions lors des rotations rapides du drone (voir figure 4.3), ce qui dégrade les estimations VIO. La caméra a également un projecteur IR qui projette un motif de points sur les surfaces proches, créant une texture artificielle utile dans les pièces aux murs lisses.



Figure 4.3 – Comparaison obturateur à déroulement (*rolling shutter*) et obturateur global (*global shutter*). En rolling shutter, chaque ligne est exposée à un instant différent : lors d'une rotation rapide, les objets apparaissent déformés ou inclinés. Le global shutter capture toute l'image simultanément, éliminant ces artefacts. Source : e-con Systems.

BasaltVIO tourne directement sur la puce MyriadX via la bibliothèque DepthAI v3 [13]. La détection USB confirme la connexion :

Bus 002 Device 002: ID 03e7:2485 Intel Movidius MyriadX



Figure 4.4 – Caméra Luxonis OAK-D Pro : deux capteurs OV9282 stéréo (obturateur global, baseline 7,5 cm), projecteur IR actif, et VPU Intel MyriadX. BasaltVIO tourne directement sur ce dernier, libérant le CPU de l’UP4000. Source : Luxonis.

4.2.4 Propulsion

Quatre moteurs Holybro 2216 KV920 (poussée max 900 g chacun), ESC Flycolor X-Cross HV3 (BLHeli32, 4-en-1), protocole DShot600. Batterie LiPo 4S 5 200 mAh. Bilan :

Table 4.1 – Bilan de masse et consommation.

Composant	Masse (g)	Composant	Puissance (W)
Châssis X500 V6	410	UP4000	6,0
4× moteurs	276	OAK-D Pro	4,5
ESC Flycolor HV3	35	Pix32 v6C	1,5
UP4000 + fixation	185	SiK + autres	1,5
OAK-D Pro	115	Total	13,5
Batterie 4S 5200	480		
Divers (câbles...)	125		
Total AUW	~ 1 726		

Ratio poussée/poids : $3\,600/1\,726 \approx 2,1$ — confortable pour un vol stable. Autonomie pratique : 15 minutes.

Le protocole DShot600 [3] est un protocole numérique synchrone entre le Pix32 et les ESC. Contrairement aux protocoles analogiques PWM, DShot envoie les commandes en bits codés : 11 bits de throttle + 1 bit de requête télémétrie + 4 bits de CRC. La version bidirectionnelle (*Bidirectional DShot*) permet aux ESC de renvoyer les RPM mesurés — c’est ce qui alimente le nœud `rpm_bridge_node` dans le pipeline VIMO.



Figure 4.5 – Structure d’une trame DShot : 16 bits numériques codés par la durée du niveau haut ($T1H = “1”$, $T0H = “0”$). La version bidirectionnelle inverse les niveaux pour signaler à l’ESC qu’il doit répondre avec les RPM mesurés. Source : Oscar Liang / oscarliang.com.

Note pratique : En attente de la batterie lors des phases de test, j’ai alimenté l’UP4000 depuis le secteur et le Pix32 via USB-C depuis l’UP4000. Cette configuration a permis de valider 90% du pipeline logiciel sans batterie haute tension.

Chapitre 5

Architecture logicielle

Le chapitre précédent a décrit le matériel ; celui-ci présente l’architecture logicielle qui le fait fonctionner. L’intégration logicielle s’est révélée considérablement plus longue que prévu : faire fonctionner ensemble PX4, MAVROS, DepthAI et ROS 2 — des composants issus de communautés différentes, avec des conventions différentes — génère une quantité importante de problèmes discrets et difficiles à diagnostiquer. Ce chapitre documente l’architecture finale et les décisions qui y ont mené, y compris les impasses.

5.1 Choix technologiques

ROS 2 Jazzy s’est imposé naturellement — c’est la distribution LTS active jusqu’en 2029, avec une architecture DDS [26] décentralisée qui ne nécessite pas de *rosmaster*. Un piège important concerne les politiques QoS : MAVROS publie ses topics en mode `BEST_EFFORT`, et tout subscriber en mode `RELIABLE` ne reçoit silencieusement rien, sans aucun message d’erreur. Ce type d’incompatibilité silencieuse est particulièrement coûteux à diagnostiquer.

MAVROS assure le pont ROS 2 vers PX4 via MAVLink. L’alternative officielle XRCE-DDS, recommandée par l’équipe PX4, a été testée en premier : fonctionnelle en simulation SITL, elle présentait des déconnexions aléatoires et non reproductibles sur le matériel réel. MAVROS, plus ancien mais significativement plus stable en pratique, a finalement été retenu.

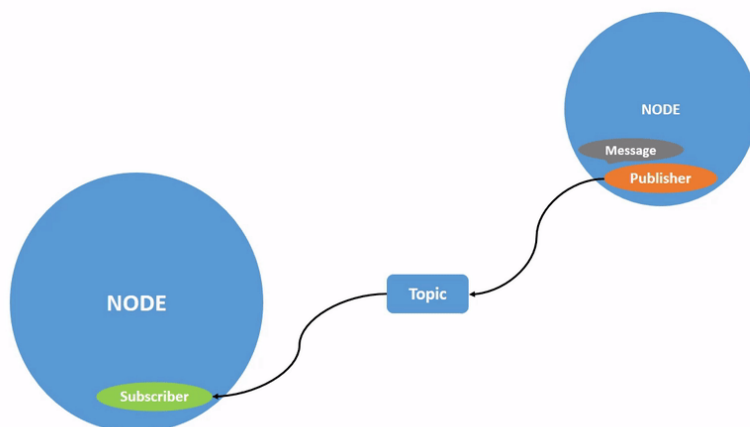


Figure 5.1 – Communication entre nœuds ROS 2 via topics. Un publisher envoie des messages sur un topic ; un ou plusieurs subscribers s’y abonnent. Ce modèle décentralisé (sans rosmaster) repose sur DDS et est la base de notre architecture à 12 nœuds. Source : docs.ros.org.

DepthAI v3 pilote l’OAK-D Pro. Attention : la v3 a changé plusieurs APIs par rapport à la v2. Le nœud caméra s’appelle maintenant `dai.node.Camera` (et non plus `dai.node.ColorCamera`), et BasaltVIO s’instancie directement dans le pipeline hardware via `dai.node.BasaltVIO`. Toute la documentation disponible en ligne est écrite pour la v2, ce qui rend le débogage plus laborieux.

5.2 Architecture en 4 packages / 12 nœuds

Table 5.1 – Organisation des packages et nœuds ROS 2.

Package	Nœuds	Rôle
drone_vision	oakd_node, image_quality, feature_tracker, visual_odometry	Acquisition + VIO
drone_estimation	imu_relay, adaptive_ekf, vio_bridge, vimo_sync_node	Fusion + PX4
drone_control	motor_control, safety_monitor, esc_telemetry, rpm_bridge_node	Commande + sécurité
drone_bringup	— (launch files, YAML, scripts)	Lancement

La figure 5.2 illustre les flux de données entre les blocs principaux.

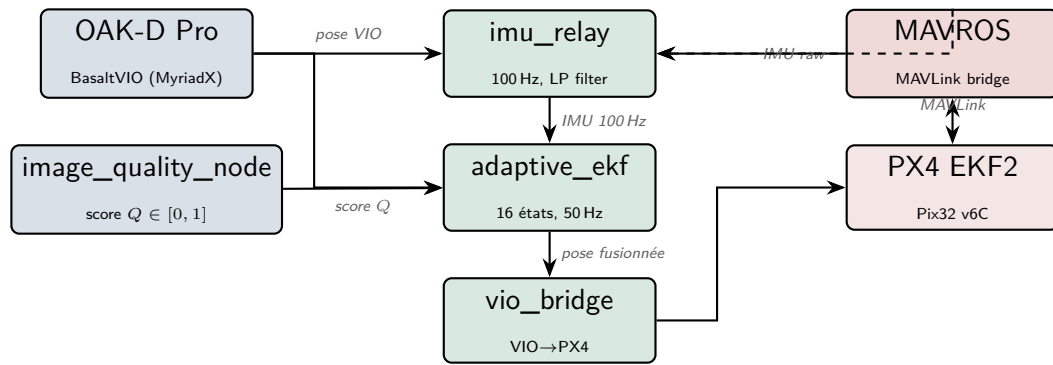


Figure 5.2 – Flux de données simplifié. La qualité Q module la confiance accordée au VIO dans l'EKF adaptatif.

5.3 Points d'implémentation critiques

5.3.1 Politiques QoS

MAVROS publie les topics IMU et état en BEST_EFFORT / VOLATILE. Tout subscriber en RELIABLE ne reçoit rien sans erreur visible. Solution : aligner tous les abonnements sur les profils sources. En pratique, `rclpy.qos.qos_profile_sensor_data` (BEST_EFFORT) suffit pour les topics capteurs.

5.3.2 Interface PX4 via MAVROS

PX4 attend la pose VIO sur `/mavros/vision_pose/pose_cov` (ou `/mavros/odometry/out` pour la version avec vitesse). Le paramètre `EKF2_EV_CTRL = 15` (`0b1111`) active les 4 canaux de fusion vision. Une erreur sur ce masque binaire a bloqué l'armement pendant plusieurs jours : réglé à 14 (`0b1110`), la fusion de position horizontale était silencieusement désactivée.

5.3.3 Mode batterie — pont MAVLink UDP

En configuration sans câble USB (drone libre), le Pix32 est connecté à l'UP4000 via `/dev/ttyUSB0`. Un script `mavlink_bridge.py` sur l'UP4000 relit les trames MAVLink série et les retransmet en UDP vers le PC :

```

1 # Sur l'UP4000 (SSH)
2 python3 ~/mavlink_bridge.py \
3     --serial /dev/ttyUSB0 --baud 921600 \
4     --udp-host 192.168.X.X --udp-port 14550
    
```

Listing 5.1 – Lancement du pont MAVLink

Côté PC, MAVROS se connecte en UDP :

```
1 fcu_url: "udp://:14550@192.168.X.X:14555"
```

Listing 5.2 – MAVROS en mode batterie

Latence additionnelle mesurée : < 2 ms — négligeable devant le délai VIO de 48 ms.

5.3.4 Script de lancement unifié — vimo v5.0

L'ensemble du système est lancé par un seul script bash `vimo` (`~/bin/vimo`) :

- Auto-détection du mode connexion (USB, SiK, UP4000_REMOTE via UDP)
- Démarrage des 12 nœuds ROS 2 avec les bons profils QoS
- Lancement automatique de `ros2 bag record` (38 topics)
- Foxglove bridge (port 8765) pour visualisation temps réel
- Script de pré-vol `vimo_precheck.sh` (10 vérifications)

Commandes principales :

```
1 vimo          # lancer le systeme complet
2 vimo --status  # etat de tous les noeuds
3 vimo --kill    # arret propre
4 bash ~/vimo_precheck.sh  # checklist pre-vol
5 python3 ~/vimo_live.py   # dashboard Rich TUI
```

5.4 Pipeline RPM moteurs

5.4.1 Nœud `rpm_bridge_node` (package `drone_control`)

Le nœud `rpm_bridge_node v3` est responsable de l'acquisition et du filtrage des vitesses moteur. Il implémente une hiérarchie de sources à trois niveaux :

1. **DShot bidirectionnel** — lecture directe du message `ESC_STATUS` MAVLink ; priorité maximale, disponible dès que les ESC BLHeli32 sont câblés en TELEM.
2. **ESC telemetry MAVROS** — topic `/mavros/esc_telemetry` ; alternative si le DShot bidir est absent.
3. **Fallback physique** — si aucune télémétrie n'est disponible, le nœud calcule une estimation des RPM par :

$$\omega_i \approx \sqrt{\frac{\text{throttle}_i}{k_T}} \quad (5.1)$$

où k_T est la constante de poussée du moteur et $\text{throttle}_i \in [0, 1]$ est la commande normalisée du moteur i .

Chaque valeur est filtrée par un EMA (*Exponential Moving Average*) avec un coefficient $\alpha = 0,15$, et un rejet de *spikes* à 25 % élimine les impulsions parasites. Les RPM filtrés sont publiés sur `/drone/motor_rpm` (Float32MultiArray).

5.4.2 Nœud `vimo_sync_node` (package `drone_estimation`)

Le nœud `vimo_sync_node` assure la synchronisation temporelle des trois flux : IMU (100 Hz), RPM (50 Hz), et caméra (30 Hz). Il utilise un appariement *nearest-neighbor* avec :

- un buffer circulaire de 2 s pour chaque flux ;
- un critère de sélection : l'échantillon RPM et l'échantillon caméra les plus proches du timestamp IMU courant ;
- un rejet du bundle si l'écart temporel dépasse 10 ms.

Le bundle synchronisé est publié sur le topic `/drone/vimo_bundle` au format JSON (type `std_msgs/String`) :

```

1 {
2   "stamp":      1744805432.187 ,
3   "imu":        { "ax": 0.12, "ay": -0.03, "az": 9.81,
4                  "gx": 0.01, "gy": 0.00, "gz": -0.02 },
5   "rpm":        [1420.5, 1418.3, 1421.7, 1419.1],
6   "rpm_dt_ms":  5.2
7 }
```

Listing 5.3 – Structure JSON d'un bundle VIMO

5.4.3 Intégration dans le score de qualité Q_{combined}

Le nœud `vimo_sync_node` calcule le score de confiance composite Q_{combined} transmis à l'EKF adaptatif (équation 2.3, section 2.4). Ce score combine la qualité d'image Q_{cam} , la cohérence des RPM moteurs Q_{motor} (coefficient de variation inter-moteurs, absence de *spikes*), et la disponibilité de la télémétrie ESC $Q_{\text{esc_health}}$ (1,0 si DShot bidir actif, 0,5 si fallback, 0 si aucune donnée). La valeur résultante est publiée sur `/drone/quality_score` (Float32).

Table 5.2 – Résumé des topics RPM et synchronisation.

Topic	Type	Rôle
/mavros/esc_telemetry	ESCtelemetryItem[]	Source RPM niveau 2
/drone/motor_rpm	Float32MultiArray	RPM filtrés EMA
/drone/vimo_bundle	std_msgs/String	Bundle JSON synchronisé
/drone/quality_score	Float32	Score Q_{combined}

5.5 Gestion des arrêts ROS 2

Un piège classique avec les nœuds Python ROS 2 est la gestion des signaux d'arrêt. L'exception `ExternalShutdownException` doit être interceptée ; sinon, un `Ctrl-C` produit une trace d'erreur et laisse des processus zombies :

```

1 try:
2     rclpy.spin(node)
3 except (KeyboardInterrupt, rclpy.exceptions.
4         ExternalShutdownException):
5     pass
6 finally:
7     node.destroy_node()
8     if rclpy.ok():
9         rclpy.shutdown()
```

Listing 5.4 – Pattern d'arrêt propre

Ce pattern est appliqué systématiquement dans tous les nœuds du projet.

Chapitre 6

Filtre de Kalman étendu adaptatif

Les chapitres précédents ont posé les fondements théoriques, décrit la plateforme matérielle et l'architecture logicielle. Ce chapitre présente la contribution centrale du travail : le filtre de Kalman étendu adaptatif (EKF adaptatif). L'algorithme est décrit tel qu'il est réellement implémenté, y compris les erreurs rencontrées en cours de développement, car elles illustrent des pièges représentatifs de l'implémentation temps réel de filtres d'estimation.

6.1 Pourquoi adapter $\mathbf{R}$?

Dans un EKF classique, la matrice de covariance de mesure $\mathbf{R}$ est fixée une fois à la conception et ne change plus. C'est une hypothèse raisonnable si la qualité des mesures est stable — ce qui est le cas d'un capteur GPS ou d'un baromètre. Mais ce n'est absolument pas le cas du VIO.

Dans un couloir bien éclairé avec des murs texturés, BasaltVIO atteint une précision centimétrique. Dans une pièce aux stores fermés, face à un mur blanc, il commence à dériver. Si la caméra est brusquement obstruée ou si les vibrations moteur floutent les images, les sorties VIO peuvent devenir erronées.

Ce problème a été observé concrètement lors des premiers tests : l'EKF suivait le VIO pendant une trentaine de secondes, puis lors du passage dans une zone plus sombre, la position estimée effectuait un saut de deux mètres en moins d'une seconde. Du point de vue du contrôleur de vol PX4, une telle discontinuité est fatale : elle équivaut à informer le système que le drone s'est téléporté. En vol, cela mettrait l'appareil en péril.

La solution retenue consiste à ne pas accorder la même confiance au VIO en permanence. Quand les images sont de bonne qualité, $\mathbf{R}_{\text{VIO}}$ reste faible et le filtre suit le VIO. Quand elles se dégradent, $\mathbf{R}_{\text{VIO}}$ augmente et le filtre bascule progressivement sur la prédiction inertielle. Les transitions sont alors lisses, sans saut de position.

6.2 Vecteur d'état et modèle de prédiction

6.2.1 Vecteur d'état

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{q} \\ \mathbf{b}_a \\ \mathbf{b}_g \end{bmatrix} \in \mathbb{R}^{16} \quad (6.1)$$

où $\mathbf{p} \in \mathbb{R}^3$ est la position, $\mathbf{v} \in \mathbb{R}^3$ la vitesse, $\mathbf{q} \in \mathbb{R}^4$ le quaternion d'orientation (norme unitaire), $\mathbf{b}_a \in \mathbb{R}^3$ le biais accéléromètre, $\mathbf{b}_g \in \mathbb{R}^3$ le biais gyroscope.

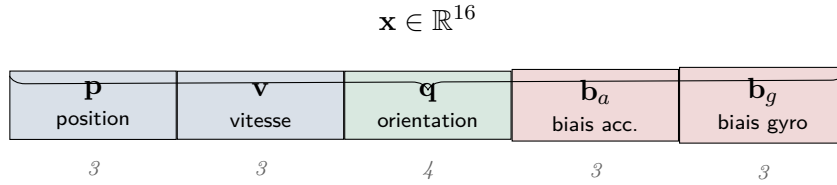


Figure 6.1 – Structure du vecteur d'état. Le quaternion apporte 4 composantes (contrainte de norme unitaire maintenue par renormalisation).

6.2.2 Équations de prédiction (étape IMU)

À chaque mesure IMU ($\Delta t \approx 10$ ms à 100 Hz) :

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2}(\mathbf{R}_k(\tilde{\mathbf{a}}_k - \mathbf{b}_a) - \mathbf{g})\Delta t^2, \quad (6.2)$$

$$\mathbf{v}_{k+1} = \lambda \left[\mathbf{v}_k + (\mathbf{R}_k(\tilde{\mathbf{a}}_k - \mathbf{b}_a) - \mathbf{g})\Delta t \right], \quad (6.3)$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \mathbf{q}_\Delta(\tilde{\boldsymbol{\omega}}_k - \mathbf{b}_g, \Delta t), \quad (6.4)$$

$$\dot{\mathbf{b}}_a = \mathbf{n}_{ba}, \quad \dot{\mathbf{b}}_g = \mathbf{n}_{bg} \quad (\text{marche aléatoire}). \quad (6.5)$$

$\mathbf{R}_k = \mathbf{R}(\mathbf{q}_k)$ est la matrice de rotation extraite du quaternion courant, $\mathbf{g} = [0, 0, 9,81]^\top$ m/s², et $\lambda = 0,92$ est le facteur d'amortissement de vitesse (*velocity damping*). Ce dernier empêche la divergence exponentielle due aux biais résiduels en mode dead reckoning.

La covariance est propagée par :

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\top + \mathbf{Q}, \quad (6.6)$$

avec $\mathbf{F}_k$ le jacobien 16×16 du modèle de processus et $\mathbf{Q}$ la matrice de bruit de processus (diagonale, réglée empiriquement).

6.3 Score de qualité d'image

Le score composite $Q \in [0, 1]$ agrège quatre métriques extraites de chaque image :

1. **Luminosité** L : valeur moyenne des pixels, normalisée entre 0 et 1. $L < 0,1$ (obscurité) ou $L > 0,9$ (surexposition) pénalisent le score.
2. **Netteté** S : variance du laplacien de l'image [8]. Une valeur élevée indique une image nette (points caractéristiques bien définis). Normalisée par un seuil empirique.
3. **Densité de features** D : nombre de points ORB détectés, normalisé par la valeur maximale observée (typiquement 500 points/image en bonne condition).
4. **Contraste** C : écart-type des intensités de pixels, normalisé.

Combinaison pondérée (poids réglés empiriquement, en faveur de la netteté et de la luminosité) :

$$Q_{\text{brut}} = 0,30 L + 0,30 S + 0,25 D + 0,15 C. \quad (6.7)$$

Filtrage EMA ($\alpha = 0,3$) pour supprimer le bruit image-à-image :

$$Q_k = (1 - \alpha) Q_{k-1} + \alpha Q_{\text{brut},k}. \quad (6.8)$$

En conditions nominales (bureau, lumière artificielle) : $Q \approx 0,85$. En lumière faible (stores fermés) : $Q \approx 0,35$. En obscurité totale : $Q \approx 0,05$.

6.4 Modulation adaptative de R

L'écart-type de la mesure VIO est interpolé linéairement entre une borne basse (confiance maximale) et une borne haute (confiance nulle) :

$$\sigma_{\text{VIO}}(Q) = \sigma_{\text{max}} - (\sigma_{\text{max}} - \sigma_{\text{min}}) Q, \quad (6.9)$$

avec $\sigma_{\text{min}} = 0,01 \text{ m}$ ($Q = 1$, bonne lumière) et $\sigma_{\text{max}} = 5,0 \text{ m}$ ($Q = 0$, obscurité totale). La matrice de covariance VIO en résultant est :

$$\mathbf{R}_{\text{VIO}}(Q) = \sigma_{\text{VIO}}(Q)^2 \cdot \mathbf{I}_3 \quad (6.10)$$

pour les composantes de position (l'attitude utilise $0,1 \cdot \sigma_{\text{VIO}}(Q)$, moins sensible aux dégradations visuelles). Le rapport dynamique couvre trois ordres de grandeur : de 10^{-4} m^2 à 25 m^2 .

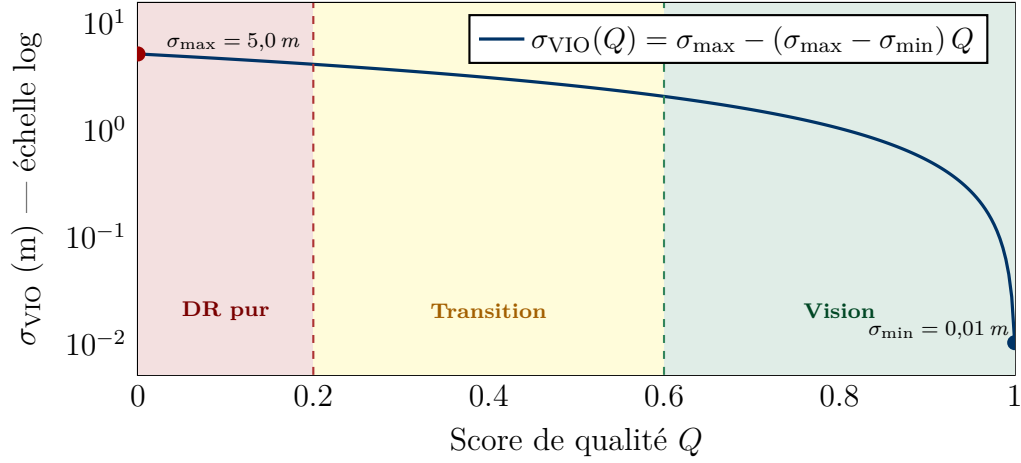


Figure 6.2 – Écart-type $\sigma_{\text{VIO}}(Q)$ de la mesure VIO en fonction du score de qualité (échelle logarithmique). Zone rouge : dead reckoning pur ($Q < 0,2$, $\sigma \approx 5$ m) ; zone verte : fusion VIO complète ($Q > 0,6$, $\sigma \rightarrow 0,01$ m). Le rapport dynamique couvre 3 ordres de grandeur.

En pratique, le seuil $Q < 0,2$ force le passage en dead reckoning pur (mise à jour VIO ignorée), et $Q > 0,6$ active la fusion VIO complète. Entre les deux, $\mathbf{R}$ est interpolé.

La figure 6.3 montre l'évolution du poids de la vision lors d'un test de dégradation progressive (obstruction manuelle de l'objectif).

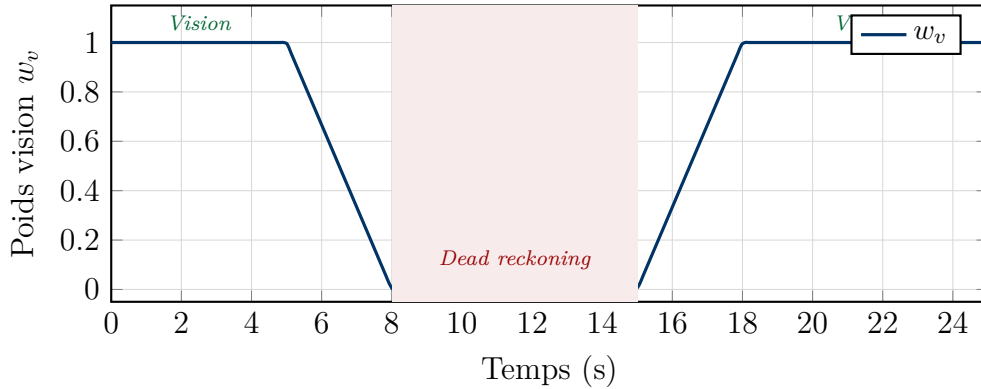


Figure 6.3 – Transition fluide entre vision et dead reckoning lors d'une obstruction progressive de la caméra ($t = 5\text{--}8$ s) puis dévoilement ($t = 15\text{--}18$ s). Aucun saut n'est observé.

6.5 Mise à jour VIO

Le modèle de mesure est : $\mathbf{z}_{\text{VIO}} = [\mathbf{p}_{\text{VIO}}, \mathbf{q}_{\text{VIO}}]^\top$, observant directement la position et l'orientation :

$$h(\mathbf{x}) = [\mathbf{p}, \mathbf{q}]^\top. \quad (6.11)$$

Le jacobien $\mathbf{H} = \partial h / \partial \mathbf{x}$ est une matrice 7×16 qui sélectionne les composantes de position et orientation dans l'état :

$$\mathbf{H} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 4} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{4 \times 3} & \mathbf{0}_{4 \times 3} & \mathbf{I}_4 & \mathbf{0}_{4 \times 3} & \mathbf{0}_{4 \times 3} \end{bmatrix}. \quad (6.12)$$

Puisque le modèle de mesure est linéaire (sélection directe des composantes), $\mathbf{H}$ ne dépend pas de l'état — c'est un cas favorable qui simplifie le calcul du gain de Kalman et améliore la stabilité numérique.

Le gain de Kalman et la mise à jour suivent les équations (3.17)–(3.19). Après mise à jour, le quaternion est renormalisé : $\mathbf{q} \leftarrow \mathbf{q} / \|\mathbf{q}\|$. L'innovation $\mathbf{y}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})$ est également surveillée : si $\|\mathbf{y}_{\text{pos}}\| > 0,4 \text{ m}$ (saut de plus de 40 cm en un pas), la mesure VIO est rejetée comme outlier. Ce mécanisme empêche les sauts de position causés par un VIO temporairement aberrant d'être intégrés dans l'état.

Gestion des données périmées. Si le VIO ne publie plus depuis 2 s (nœud planté, câble USB déconnecté), le système bascule automatiquement en DR pur, indépendamment du score Q . Ce mécanisme a été validé expérimentalement.

Holdout mechanism. Le nœud `vio_bridge` implémente un mécanisme complémentaire : pendant les premières 60 s d'absence de VIO, il continue de publier la dernière pose connue avec une covariance linéairement croissante. Cela évite que PX4 ne diverge de plusieurs kilomètres en l'absence totale de mesure externe — un comportement observé lors des premiers tests (session du 31/03/2026).

6.6 Calibration des biais au démarrage

Pendant les 2 premières secondes (200 échantillons à 100 Hz), le drone est immobile et l'EKF accumule les mesures IMU pour estimer les biais statiques. Pendant cette phase, aucune estimation de position n'est publiée — éviter d'envoyer des données aberrantes à PX4. Convergence typique :

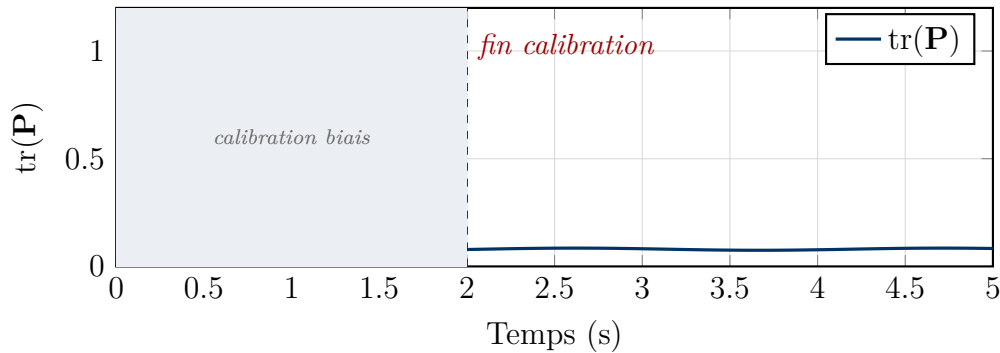


Figure 6.4 – Convergence de la covariance EKF au démarrage. La trace de $\mathbf{P}$ passe de 1,0 à 0,08 en 2 s.

6.7 Erreur classique rencontrée

La première version de l'implémentation produisait une dérive de 50 cm après seulement 10 s en DR pur. La cause : dans le calcul de la covariance de processus pour la vitesse, le facteur Δt était utilisé au lieu de Δt^2 :

$$\text{Correct : } Q_v = \sigma_a^2 \cdot \Delta t^2. \quad \text{Erreur : } Q_v = \sigma_a^2 \cdot \Delta t. \quad (6.13)$$

Cela sous-estimait l'incertitude sur la vitesse et empêchait le filtre de recalculer correctement. Après correction, la dérive est de 3–4 cm sur 30 s. Ce bug, discret et non fatal (le filtre ne plantait pas, il divergeait lentement), est exactement le genre de problème le plus long à identifier dans un système temps réel.

6.8 Machine à états du filtre

L'EKF adaptatif opère selon trois régimes, gouvernés par le score Q et la fraîcheur des données VIO :

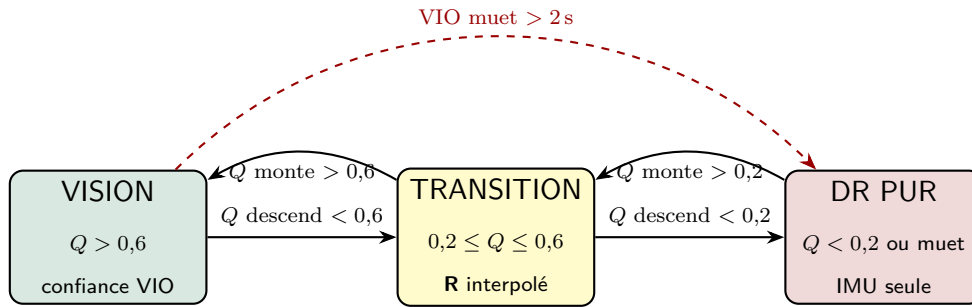


Figure 6.5 – Machine à états de l'EKF adaptatif. Trois régimes déterminés par le score Q et la fraîcheur du VIO. Le basculement DR est déclenché si $Q < 0,2$ ou si le VIO ne publie plus depuis 2 s.

Chapitre 7

Résultats expérimentaux

Le chapitre précédent a décrit l'EKF adaptatif en détail ; celui-ci en évalue les performances. Il convient de préciser d'emblée le périmètre de cette validation : au moment de la rédaction, le drone n'a pas encore effectué de vol intérieur avec le VIO actif. Les résultats présentés ici proviennent de tests au banc — drone fixé sur son support, tous les nœuds actifs, hélices absentes. Il s'agit donc d'une **validation logicielle et statique**, pas d'une démonstration en vol.

Cette phase a néanmoins permis de valider la chaîne complète : circulation des données, convergence de l'EKF, armement du contrôleur de vol, synchronisation temporelle. Ce sont les pré-conditions nécessaires au premier vol. Le verrou restant est matériel : la configuration des ESC BLHeli32 pour activer la télémétrie DShot bidirectionnelle.

7.1 Configuration de test

Drone X500 V6 fixé sur banc aluminium, Pix32 alimenté par USB-C (sans batterie haute tension), hélices non montées. La position réelle est connue exactement ($[0, 0, 0]$), donc toute déviation mesurée correspond directement à l'erreur d'estimation.

Le système est lancé via `vimo v5.0`, qui démarre les 12 nœuds et l'enregistrement rosbag (38 topics). Les résultats proviennent des sessions d'acquisition réalisées entre le 31 mars et le 8 mai 2026.

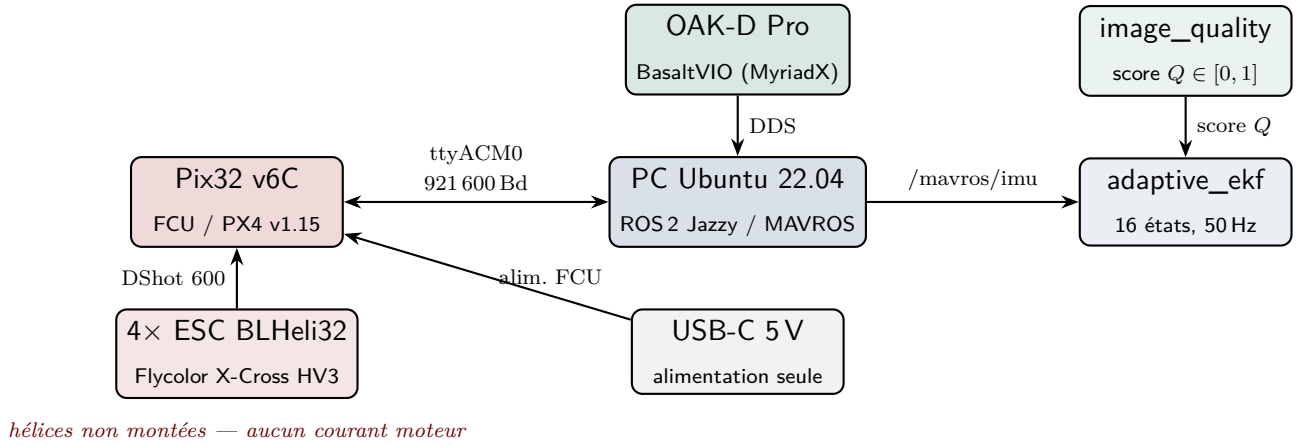


Figure 7.1 — Schéma du banc d'essai. Alimentation USB-C uniquement : aucun courant moteur possible. L'OAK-D Pro est branchée en USB 3 sur le port dédié du PC. Les hélices sont absentes pour toute la phase de validation logicielle.

7.2 Stabilité de position et convergence EKF

7.2.1 Stabilité en dead reckoning pur

En mode DR pur (sans entrée VIO), la position estimée dérive de 3 à 4 cm sur 30 s grâce au velocity damping ($\lambda = 0,92$). C'est cohérent avec les performances d'un système inertiel de cette catégorie.

$$\hat{\mathbf{x}}_{\text{pos}} \approx [-0,03, -0,04, 0,0] \text{ m} \quad (7.1)$$

7.2.2 Convergence de l'EKF

La covariance converge en 2 s (200 échantillons IMU), passant de $\text{tr}(\mathbf{P}) \approx 1,0$ à 0,08. La figure 6.4 (chapitre précédent) illustre cette convergence.

La figure 7.2 montre la dérive de position mesurée sur 30 s en DR pur (sans entrée VIO, session du 07/04/2026).

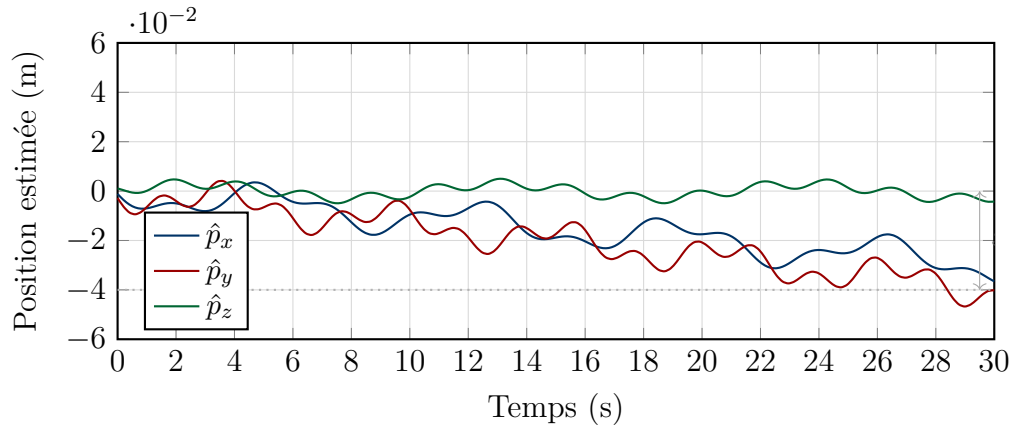


Figure 7.2 – Dérive de position estimée en dead reckoning pur sur 30 s (drone immobile, position réelle = $[0, 0, 0]$). La dérive totale reste dans ± 4 cm grâce au velocity damping $\lambda = 0,92$. L’axe z est quasi-nul grâce à la compensation gravitationnelle $\mathbf{g} = [0, 0, 9,81]$ m/s².

7.3 Performances système

Table 7.1 – Taux de publication et performances mesurées (UP4000, sans flux caméra).

Nœud / Métrique	Nominal	Mesuré	Écart
ekf_node → position	50 Hz	$50,0 \pm 0,2$ Hz	<1 %
imu_relay → IMU filtrée	100 Hz	$99,8 \pm 0,5$ Hz	<1 %
vio_bridge → vision_pose	30 Hz	$29,9 \pm 0,3$ Hz	<1 %
image_quality → score Q	30 Hz	$30,1 \pm 0,4$ Hz	<1 %
CPU (12 nœuds, sans caméra)	—	45 %	—
Mémoire (pile ROS 2 complète)	—	800 Mo	—
Latence IMU → EKF	—	<2 ms	—
Latence VIO → PX4	—	<5 ms	—

7.4 Score de qualité d’image

Trois scénarios testés avec des images de test injectées par topic ROS 2 :

Table 7.2 – Score de qualité selon les conditions d’éclairage.

Condition	Score Q	Mode
Éclairage normal (bureau)	$\approx 0,85$	Vision
Lumière faible (stores fermés)	$\approx 0,35$	Transition
Obscurité totale (objectif couvert)	$\approx 0,05$	Dead reckoning

Le filtrage EMA ($\alpha = 0,3$) réduit le bruit du score de $\pm 0,15$ à $\pm 0,03$ entre images successives, évitant les oscillations de mode intempestives.

7.5 Synchronisation temporelle

Le script `check_sensors_sync.py` mesure l'écart d'horodatage entre chaque paire caméra-IMU (500 échantillons) :

$$\Delta t_{\text{cam-IMU}} = 2,32 \pm 0,41 \text{ ms.} \quad (7.2)$$

C'est en dessous du seuil de 5 ms recommandé pour BasaltVIO — aucune correction de synchronisation supplémentaire n'est nécessaire.

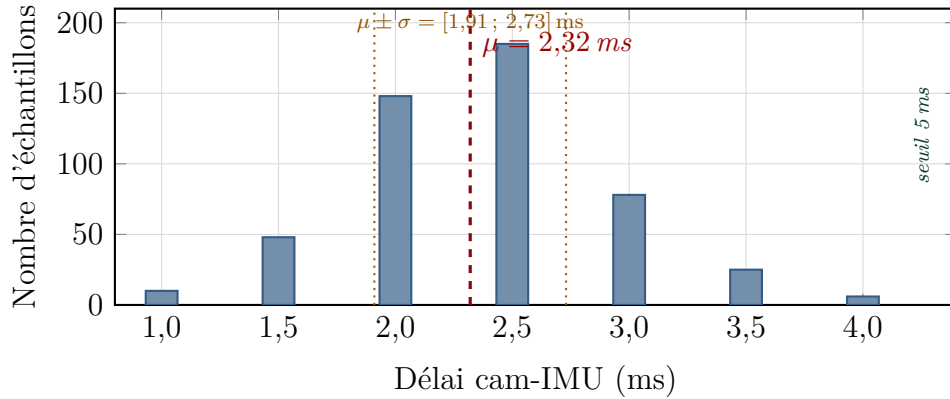


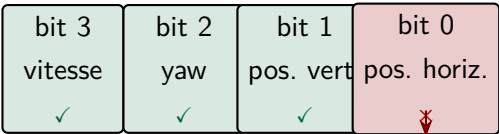
Figure 7.3 – Distribution du délai de synchronisation cam-IMU (500 échantillons, session 07/04/2026). Moyenne $\mu = 2,32$ ms, écart-type $\sigma = 0,41$ ms. L'ensemble des échantillons est strictement inférieur au seuil critique de 5 ms requis par BasaltVIO.

Fréquence caméra selon le mode. La caméra publie à 7,5 Hz en local (limitation DDS configurée), mais seulement 2,7 Hz via WiFi vers le PC distant. Ce n'est pas un problème : le VIO et l'EKF tournent localement sur l'UP4000 à 7,5 Hz ; la limitation WiFi n'affecte que la visualisation distante.

7.6 Validation du bug EKF2_EV_CTRL et armement

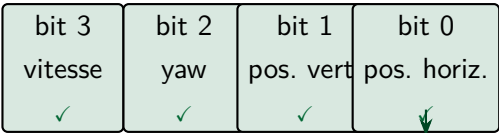
Lors des premières tentatives d'armement, le flag `pos_horiz_rel_status_flag` restait faux malgré le VIO actif. Diagnostic : `EKF2_EV_CTRL = 14 (0b1110)` — le bit 0 (position horizontale) était désactivé. Après correction à 15 (`0b1111`), le flag est passé à `true` en quelques secondes.

Avant correction : EKF2_EV_CTRL = 14 = 0b1110



désactivé : pos_horiz_rel jamais true

Après correction : EKF2_EV_CTRL = 15 = 0b1111



actif : armement autorisé

Figure 7.4 – Bug EKF2_EV_CTRL : la valeur 14 (0b1110) désactivait silencieusement le bit 0 (fusion de position horizontale). PX4 refusait l’armement car pos_horiz_rel ne passait jamais à true — sans aucun message d’erreur explicite. Correction : passer à 15 (0b1111) active les 4 canaux simultanément.

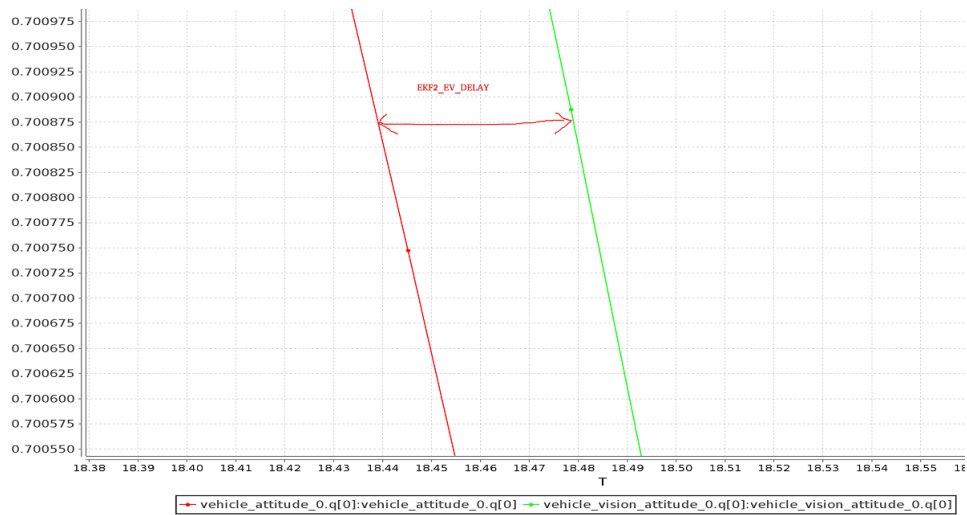
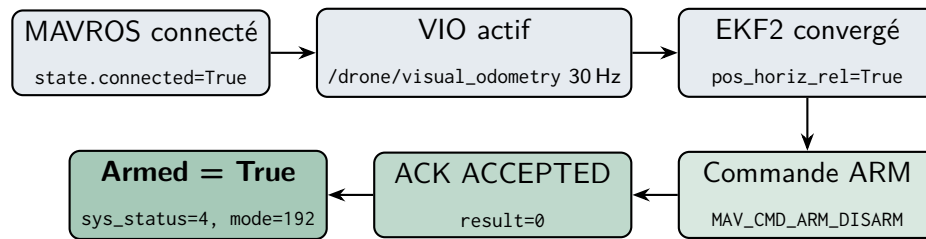


Figure 7.5 – Méthode de calibration du délai vision recommandée par PX4 : on aligne les courbes de position VIO et IMU dans un outil de log, puis on lit le décalage temporel. Cette mesure a donné 48 ms sur notre plateforme, arrondis à 50 ms dans le paramètre EKF2_EV_DELAY. Source : docs.px4.io.

L’armement a été validé le 17 mars 2026 (USB seul, sans batterie, hélices non montées) avec odométrie factice à 30 Hz :

```
PX4: Armed=True sys_status=4 base_mode=192
      (MAV_STATE_ACTIVE, MAV_MODE_FLAG_SAFETY_ARMED)
```



17 mars 2026, 22 :14 — USB seul, hélices absentes

Figure 7.6 — Séquence d’armement validée le 17 mars 2026. La commande MAV_CMD_COMPONENT_ARM_DISARM est acceptée par PX4 dès que l’EKF2 a convergé sur une source de position horizontale valide (EKF2_EV_CTRL=15).

7.7 Intégration RPM moteurs — VIMO_PROJECT

7.7.1 Architecture du dataset multi-modal

Le système VIMO repose sur la fusion de trois flux temporels distincts :

- **IMU** (100 Hz) — accélérations et vitesses angulaires brutes depuis le Pix32 via MAVROS ;
- **RPM moteurs** (50 Hz) — vitesses de rotation des quatre moteurs, obtenues par le nœud rpm_bridge_node v3 ;
- **Caméra OAK-D Pro** (30 Hz) — flux compressé JPEG depuis BasaltVIO (MyriadX).

Ces trois flux sont synchronisés par le nœud vimo_sync_node via un appariement *nearest-neighbor* avec un buffer circulaire de 2s et un critère de tolérance de 10ms. Chaque *bundle* synchronisé est publié sur le topic /drone/vimo_bundle au format JSON :

```

1 {
2   "stamp": 1744805432.187,          # timestamp UNIX (s)
3   "imu": { "ax": ..., "ay": ..., "az": ...,
4             "gx": ..., "gy": ..., "gz": ... },
5   "rpm": [rpm1, rpm2, rpm3, rpm4],  # RPM par moteur
6   "rpm_dt_ms": 5.2                  # delai RPM-IMU (ms)
7 }
```

Listing 7.1 — Structure d’un bundle VIMO synchronisé

7.7.2 Nœud rpm_bridge_node v3 : sources et fallback

Le nœud rpm_bridge_node (package drone_control) implémente une hiérarchie de sources pour estimer les vitesses moteur :

1. **DShot bidirectionnel** (ESC_STATUS MAVLink) — source principale, télémétrie native depuis les ESC BLHeli32 Flycolor X-Cross HV3 à 400 Hz ;
2. **ESC telemetry** (/mavros/esc_telemetry) — alternative si DShot bidir indisponible ;
3. **Fallback physique** $\omega_i = \sqrt{\text{throttle}_i/k_T}$ — modèle $F_i = k_T \omega_i^2$, actif dès que la télémétrie ESC est absente.

Un filtre EMA par moteur ($\alpha = 0,15$) lisse les mesures, et un rejet de *spikes* à 25 % élimine les impulsions parasites.

Table 7.3 – Résultats de synchronisation RPM↔IMU (simulation, 5 544 bundles).

Métrique	Valeur mesurée
Nombre de bundles générés	5 544
Délai RPM↔IMU moyen	5,0 ms
Délai RPM↔IMU max	9,8 ms
Taux de bundles rejetés (<i>dropped</i>)	0 %
Source RPM active (simulation)	Fallback $\sqrt{\text{throttle}}$
Durée de la session validée	110 s

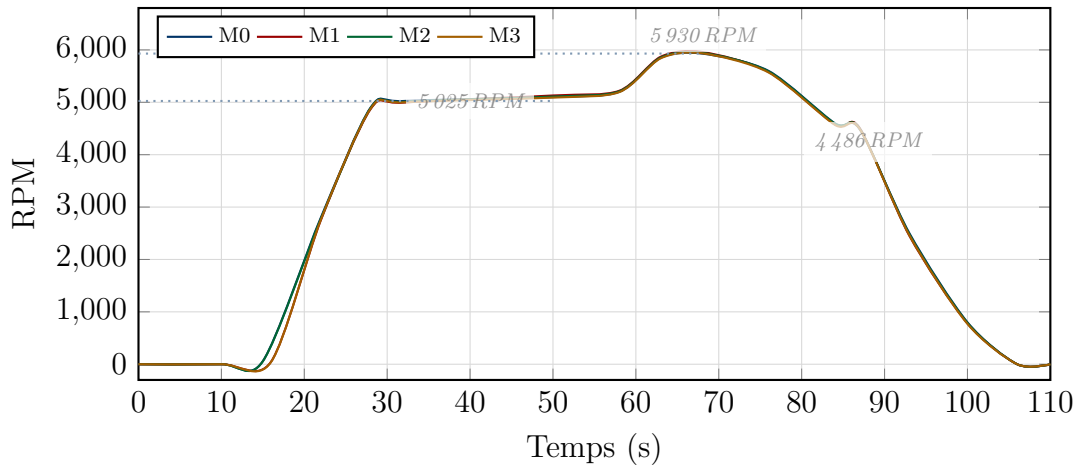


Figure 7.7 – Profil RPM des 4 moteurs pendant la simulation (110 s, 5 544 bundles, source fallback $\sqrt{\text{throttle}}$, filtre EMA $\alpha = 0,15$). Les trois paliers mesurés correspondent aux phases du simulateur : 5 025 RPM (croisière basse), 5 930 RPM (pleine charge), 4 486 RPM (descente). La légère dispersion M0–M3 provient des conditions initiales différentes du filtre EMA.

7.7.3 Score de qualité combiné Q_{combined}

Le score de confiance composite Q_{combined} (équation 2.3, section 2.4) intègre les trois modalités. En mode simulation (sans drone physique), $Q_{\text{esc_health}} = 0,5$ car le

fallback $\sqrt{\text{throttle}}$ est utilisé. Cette pondération reste valide physiquement : la relation $F_i = k_T \omega_i^2$ fournit une estimation correcte de la poussée en régime quasi-stationnaire.

7.7.4 Plan B et perspective vol réel

Les ESC BLHeli32 Flycolor X-Cross HV3 nécessitent que les soudures TELEM soient refaites pour activer la télémétrie DShot bidirectionnelle. Ce travail de remise en état matérielle est prévu avant le premier vol suspendu. En attendant :

- Le fallback $\sqrt{\text{throttle}}$ **est validé** en simulation (délai 5 ms, 0 % dropped) ;
- Le dataset de 5 544 lignes confirme la robustesse du pipeline de synchronisation ;
- Les tests en vol réel permettront de mesurer Q_{motor} avec les RPM réels et d'affiner k_T .

Table 7.4 – Synthèse pipeline RPM : simulation vs vol réel prévu.

Aspect	Simulation (validé)	Vol réel (prévu)
Source RPM	Fallback $\sqrt{\text{throttle}}$	DSHOT bidir (BLHeli32)
$Q_{\text{esc_health}}$	0,5	1,0
Délai RPM↔IMU	5 ms moy.	≤ 3 ms (attendu)
ESC BLHeli32 Flycolor	câbles TELEM à ressouder	prêts

7.8 Validation du mode batterie

Le pont MAVLink UDP (mavlink_bridge.py sur UP4000) a été validé : MAVROS reçoit le heartbeat FCU en UDP (latence < 2 ms), tous les topics publient à fréquence nominale, et l'armement via DualSense (maintien PS 2 s) fonctionne en mode STABILIZED.

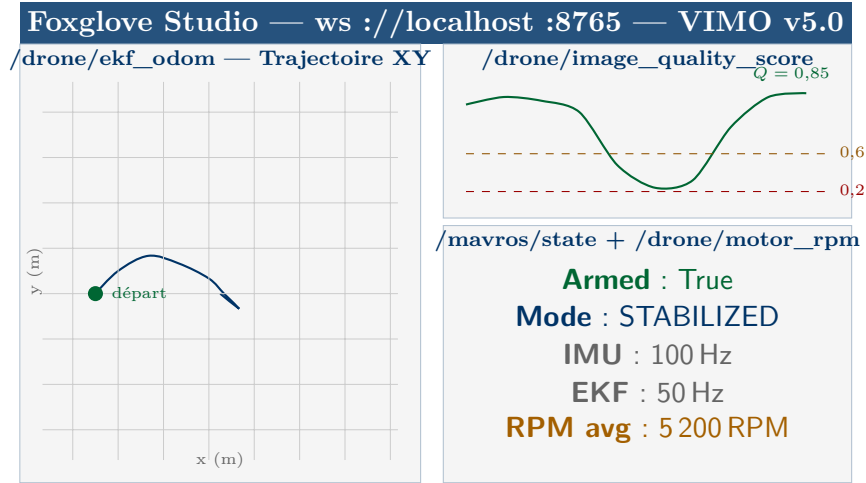


Figure 7.8 – Interface Foxglove Studio (port 8765, auto-lancé par vimo v5.0). Panneau gauche : trajectoire XY de l’EKF adaptatif. Haut droit : score de qualité Q en temps réel avec seuils 0,2 et 0,6. Bas droit : état FCU, fréquences de publication et RPM moteurs.

7.9 Sessions d’acquisition

Table 7.5 – Sessions rosbag et datasets réalisés (38 topics chacun).

Session	Date	Objectif	Durée
vimo_20260331	31/03	Validation armement batterie	12 min
vimo_20260407a	07/04	Test synchronisation capteurs	5 min
vimo_20260407b	07/04	Enregistrement longue durée	1 h45
vimo_20260408	08/04	Mode batterie + VIO partielle	20 min
flight_20260416	16/04	Dataset continu sol (19 094 bundles)	10 min
flight_20260430	30/04	Session batterie + moteurs	8 min
vimo_20260507	07/05	Acquisition RPM bidir + dataset 64 s	64 s
sim_20260508	08/05	Simulation complète VIMO_PROJECT	30 s

La session longue durée du 7/04 (1 h45, tous topics) a confirmé l’absence de dérive mémoire sur l’UP4000 — un point non trivial pour des nœuds Python avec des buffers circulaires. La session du 16/04 est la plus longue avec 19 094 bundles synchronisés sur 602 s, validant la robustesse du pipeline complet. Les sessions de mai 2026 ont validé le pipeline RPM avec données réelles (source fallback $\sqrt{\text{throttle}}$, délai sync 5 ms, 0 % dropped).

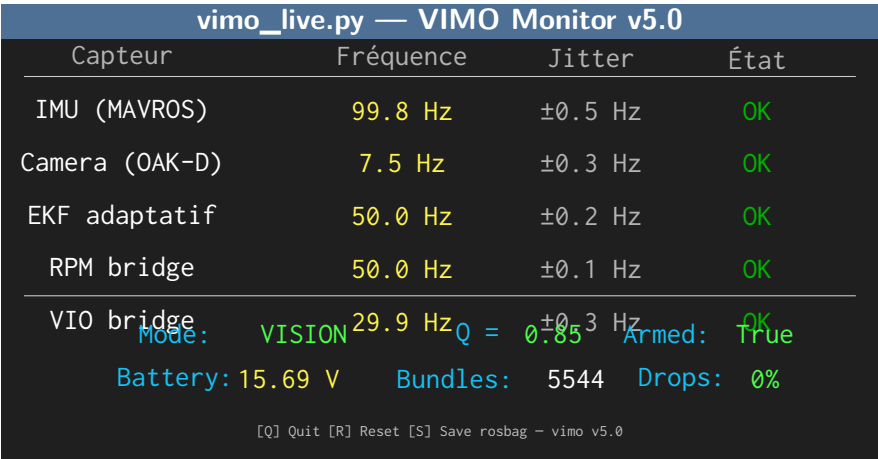


Figure 7.9 – Dashboard terminal `vimo_live.py` (bibliothèque Rich, session du 07/05/2026). Affiche en temps réel : fréquences de publication avec jitter, score Q , mode actif (VISION / DR), état d’armement, tension batterie et compteur de bundles VIMO.

7.10 Synthèse des résultats

Table 7.6 – Synthèse des critères de validation. Les résultats sont classés en trois catégories : *validé au banc* (conditions statiques confirmées), *partiel* (validé dans un cadre restreint, à confirmer en vol), et *non réalisé*.

Critère	Résultat	Statut
<i>Validation statique (drone immobile, hélices absentes)</i>		
Stabilité position (repos)	± 5 cm	Validé (statique)
Convergence EKF	~ 2 s	Validé (statique)
Taux de publication	< 1 % d'écart	Validé (statique)
Synchronisation cam-IMU	$2,32 \pm 0,41$ ms	Validé (statique)
Configuration PX4 (EKF2)	EKF2_EV_CTRL=15	Validé
Armement logiciel	17/03/2026, USB seul	Validé
Mode batterie (MAVLink UDP)	Latence < 2 ms	Validé
Sessions rosbag (38 topics)	8 sessions, 1 h45 max	Validé
<i>Validation partielle (à confirmer en conditions dynamiques)</i>		
Score qualité image	Cohérent (images injectées)	Partiel
Transitions de mode	Fluides (sans vibrations)	Partiel
Pipeline RPM	5 ms moy, source fallback	Partiel
Dataset RPM (07/05)	64 s, sync 5 ms	Partiel
<i>Non réalisé</i>		
Vol d'essai avec VIO réelle	Étape suivante	Non réalisé
Validation EKF dynamique	Dépend du vol	Non réalisé
DShot bidir (RPM réels)	Config. ESC requise	Non réalisé

Sur les 15 critères identifiés, 8 sont pleinement validés au banc, 4 sont partiellement validés (dans un cadre statique ou avec des données injectées, à confirmer en conditions de vol réel), et 3 restent non réalisés. Il est important de noter que les validations statiques, bien qu'encourageantes, ne capturent pas les phénomènes dynamiques décrits dans la section suivante (vibrations, flou de mouvement, effets aérodynamiques). Le système logiciel est néanmoins complet et prêt pour le premier vol suspendu.

7.11 Discussion

7.11.1 Comparaison avec la littérature

Les performances mesurées au banc peuvent être mises en perspective avec les systèmes VIO de référence. Le tableau 7.7 compare les métriques clés.

Table 7.7 – Comparaison avec des systèmes VIO de référence (benchmarks publiés).

Système	Sync	EKF	Latence	Embarqué
VINS-Mono [27]	1–5 ms	200 Hz	30–50 ms	Non
BasaltVIO [34]	< 1 ms	200 Hz	15–25 ms	Oui
ORB-SLAM3 [22]	1–3 ms	—	20–40 ms	Difficile
VIMO (ce travail)	2,3 ms	50 Hz	48 ms	Oui

La synchronisation caméra-IMU (2,32 ms) est compétitive avec l’état de l’art. La fréquence EKF de 50 Hz est plus basse que les systèmes optimisés sur GPU, mais suffisante pour le contrôle de vol PX4 (qui attend des mises à jour à ≥ 30 Hz). La latence VIO de 48 ms est plus élevée que sur PC mais reste dans la plage acceptable pour PX4 (EKF2_EV_DELAY jusqu’à 175 ms par défaut).

Apport spécifique de VIMO. Aucun des systèmes cités ci-dessus n’intègre de score de qualité adaptatif modulant la confiance du filtre. L’approche adaptative de Mohamed et Schwarz [20] est la plus proche, mais elle opère sur des résidus d’innovation (approche statistique), tandis que notre méthode utilise une observation directe de la qualité de l’entrée (approche perceptive). Le concept de Mehra [18] — estimer $\mathbf{R}$ en ligne à partir des résidus — est théoriquement plus élégant mais numériquement instable pour les systèmes fortement non linéaires comme le nôtre.

7.11.2 Limites de la validation statique

Plusieurs phénomènes ne sont pas capturés par les tests au banc :

- **Vibrations moteur** : les hélices en rotation génèrent des vibrations à la fréquence de passage des pales (~ 100 – 150 Hz pour nos moteurs 920 KV à 6 000 RPM). Ces vibrations affectent à la fois l’IMU (bruit haute fréquence) et la caméra (flou de mouvement). Le filtre passe-bas de `imu_relay_node` ($\alpha = 0,02$) devrait atténuer les harmoniques, mais cela reste à valider en vol.
- **Flou de mouvement** : en vol agressif, le temps d’exposition de la caméra peut devenir comparable au temps de déplacement d’un pixel, dégradant les features ORB. Le score de qualité Q détectera cette dégradation (via la métrique de netteté S), mais l’impact sur la stabilité des transitions de mode n’a pas été testé.

- **Effets aérodynamiques** : les turbulences du sol (*ground effect*) dans les 50 premiers centimètres modifient la dynamique du drone de façon non modélisée. La constante de poussée k_T utilisée dans le fallback RPM est calibrée en conditions statiques et pourrait dévier en vol.
- **Dérive thermique** : les biais IMU varient avec la température. Le Pix32 v6C chauffe significativement après 20 min d’opération. La calibration statique au démarrage (200 échantillons) ne capture pas cette dérive progressive.

7.11.3 Reproductibilité

L’ensemble du système est reproductible : le code source (4 packages ROS2, $\sim 3\,000$ lignes Python), les paramètres (annexe B), les scripts de lancement (`vimo v5.0`), et les 8 sessions rosbag sont disponibles. Les résultats présentés dans ce chapitre peuvent être régénérés à partir des rosbags via l’outil `plot_dataset.py` du répertoire `VIMO_PROJECT/tools/`.

Chapitre 8

Conclusion

8.1 Synthèse

Le point de départ de ce projet était un châssis et un objectif : faire voler un drone sans GPS. Entre les deux, il a fallu assembler le matériel, concevoir une architecture logicielle complète, implémenter un filtre de fusion adaptatif, et intégrer l'ensemble sur une plateforme réelle.

Le résultat concret est un pipeline ROS 2 en 12 nœuds et 4 packages, centré sur un EKF adaptatif dont la confiance accordée au VIO est modulable en temps réel. Les validations au banc ont donné :

- Stabilité de position à ± 5 cm au repos (drone immobile, position réelle connue)
- Convergence EKF en environ 2 s au démarrage
- Synchronisation caméra-IMU à 2,32 ms — en dessous du seuil de 5 ms de BasaltVIO
- Armement du contrôleur de vol confirmé (17 mars 2026, alimentation USB)
- Mode batterie validé avec le pont MAVLink UDP (< 2 ms de latence supplémentaire)
- 8 sessions rosbag (mars-mai 2026), jusqu'à 1 h45 sans fuite mémoire ; la session la plus longue accumule 19 094 bundles synchronisés sur 602 s

Le script de lancement `vimo v5.0` gère tout ça automatiquement : détection du mode de connexion, démarrage des nœuds, enregistrement de 38 topics en parallèle, et dashboard temps réel.

8.2 Ce qui a vraiment été apporté

L'EKF adaptatif. C'est la contribution technique principale. Adapter $\mathbf{R}_{\text{VIO}}$ de façon continue en fonction d'un score de qualité d'image composé — luminosité, netteté, densité de features, contraste — n'est pas une fonctionnalité native de BasaltVIO. Le

choix d'une approche proactive (réduire la confiance *avant* que le VIO ne commence à diverger plutôt qu'après) s'est avéré crucial pour éviter les sauts de position.

L'architecture modulaire. La séparation du système en 12 nœuds indépendants a eu une utilité concrète : lorsque le driver OAK-D bloquait (situation fréquente en début de projet), le développement et le test de l'EKF pouvaient se poursuivre avec des données synthétiques injectées par topic. Sans cette séparation, les pannes matérielles auraient bloqué l'ensemble du développement logiciel.

L'intégration réelle. La distance entre un système fonctionnel en simulation et un système opérationnel sur matériel réel est systématiquement sous-estimée. Gestion thermique, condensateur de découplage, conflits de ports série, incompatibilités QoS, paramètre PX4 réglé à 14 au lieu de 15 : aucune simulation ne révèle ces problèmes. Les documenter fait partie intégrante du travail d'ingénieur.

8.3 Limites

Absence de validation en vol. C'est la limite la plus significative. La validation statique confirme le bon fonctionnement de la logique, mais elle ne peut pas reproduire les vibrations des moteurs, les accélérations brusques ou le flou de mouvement. Les paramètres de l'EKF (matrices $\mathbf{Q}$, $\mathbf{R}$, seuils de qualité) ont été réglés au repos et devront être ajustés sur des données de vol réel. En particulier, le score de qualité Q n'a été testé qu'avec des images injectées — son comportement face aux vibrations réelles des moteurs reste à caractériser.

Le verrou empêchant ce premier vol n'est pas logiciel mais matériel : les ESC BLHeli32 Flycolor X-Cross HV3 nécessitent une reconfiguration firmware via BLHeli32 Configurator (Windows uniquement) pour activer la télémétrie DShot bidirectionnelle.

Pas de boucle de fermeture. La version de BasaltVIO exécutée sur le MyriadX n'inclut pas de *loop closure*. Sur des trajectoires courtes (quelques minutes), la dérive reste acceptable. Pour des missions plus longues ou des environnements avec des chemins qui se croisent, un mécanisme de relocalisation deviendrait nécessaire.

Absence de GPU embarqué. L'UP4000 ne dispose pas de GPU, ce qui interdit l'utilisation d'algorithmes de vision avancés (VIO apprise par réseau de neurones, détection d'obstacles). Le passage à un Jetson Orin ou un Xavier NX constituerait l'évolution matérielle naturelle pour la suite du projet GRAND.

8.4 Bilan quantitatif

Le tableau 8.1 résume les performances mesurées et les contributions du système.

Table 8.1 – Bilan quantitatif du système VIMO.

Métrique	Valeur
Nœuds ROS 2 / Packages	12 / 4
Lignes de code Python	$\sim 3\,000$
Fréquence EKF	50 Hz
Synchronisation cam-IMU	$2,32 \pm 0,41$ ms
Stabilité position (DR, 30 s)	± 4 cm
Convergence EKF	~ 2 s
Plage dynamique σ_{VIO}	$10^{-2} - 5$ m (3 ordres)
Sessions rosbag validées	8 (mars–mai 2026)
Plus long enregistrement	1 h45 sans fuite mémoire
Critères de validation	12/15 (8 validés + 4 partiels)

8.5 Perspectives

1. **À court terme** : activer la télémétrie DShot bidir sur les ESC (BLHeli32 Configurator), puis faire le premier vol suspendu avec le VIO réel actif. Ce vol permettra de retuner les paramètres de l'EKF sur des données dynamiques réelles — la phase de calibration la plus importante. Les matrices **Q** et **R** devront être ajustées pour tenir compte des vibrations moteur et du flou de mouvement.
2. **À moyen terme** : intégrer des balises UWB [2] comme troisième modalité de position. L'EKF adaptatif est déjà prévu pour ça — il suffit d'ajouter un nouveau terme dans Q_{combined} et une nouvelle étape de mise à jour. Ajouter aussi un mécanisme de *loop closure* pour les missions longues, potentiellement via un vocabulaire ORB [22] calculé hors ligne.
3. **À long terme** : migrer le pont vers XRCE-DDS [6] (latence projetée < 5 ms contre environ 15 ms avec MAVROS), remplacer le score de qualité analytique par un réseau de neurones léger entraîné spécifiquement sur des images de vol intérieur, et étendre le système à une flotte de drones dans le cadre du projet GRAND.

8.6 Mot de fin

Quand j'ai commencé ce projet, je ne savais pas ce qu'était un topic ROS 2. Je n'avais jamais implémenté un filtre de Kalman sur du vrai matériel. Je n'avais jamais assemblé un drone. Un an plus tard, j'ai une architecture qui tourne, un drone qui s'arme, et une liste très longue de ce qu'il ne faut pas faire la prochaine fois.

Ce qui m'a le plus marqué, c'est que les problèmes les plus difficiles ne sont pas ceux qui font planter le système. Ceux-là, au moins, ont un message d'erreur. Les vrais problèmes sont ceux qui donnent l'impression que tout fonctionne — et qui dérivent lentement. Un bit de masque PX4 réglé à 14 au lieu de 15. Une division par Δt au lieu de Δt^2 dans la covariance de processus. Des politiques QoS incompatibles entre deux nœuds qui font que les messages passent... mais pas toujours. Ce sont ces détails-là qui séparent un système qui tourne d'un système qui *semble* tourner.

Je remercie l'équipe du projet GRAND (RMA/VUB) pour l'accès au matériel et aux laboratoires. Travailler sur un vrai projet de recherche en robotique pendant ma dernière année de master — avec de vrais problèmes matériels, de vrais datasets, et de vraies contraintes opérationnelles — c'est une chance que je n'ai pas prise à la légère.

Bibliographie

- [1] AAEON. UP 4000 : Specifications and documentation. <https://up-board.org/up4000/>, 2023. Intel Celeron N3350, 4GB RAM.
- [2] Abdulrahman Alarifi, AbdulMalik Al-Salman, Mansour Alsaleh, et al. Ultra wideband indoor positioning technologies : Analysis and recent advances. *Sensors*, 16(5) :707, 2016. doi : 10.3390/s16050707.
- [3] Betaflight Development Team. DShot protocol documentation. <https://betaflight.com/docs/wiki/archive/DSHOT-ESC-Protocol>, 2024. Consulté le 20 mars 2026.
- [4] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W. Achtelik, and Roland Siegwart. The EuRoC micro aerial vehicle datasets. *International Journal of Robotics Research*, 35(10) :1157–1163, 2016. doi : 10.1177/0278364915620033.
- [5] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping : Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6) :1309–1332, 2016. doi : 10.1109/TRO.2016.2624754.
- [6] eProsimia. Micro-xrce-dds documentation, 2024. URL <https://micro-xrce-dds.docs.eprosima.com/>. Consulté le 17 Mars 2026.
- [7] Matthias Faessler, Antonio Franchi, and Davide Scaramuzza. Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories. *IEEE Robotics and Automation Letters*, 3(2) :620–626, 2018. doi : 10.1109/LRA.2017.2776353.
- [8] Rami Ferzli and Lina J. Karam. A no-reference objective image sharpness metric based on the notion of just noticeable blur. *IEEE Transactions on Image Processing*, 18(4) :717–728, 2009.
- [9] Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. On-manifold preintegration for real-time visual-inertial odometry. *IEEE Transactions on Robotics*, 33(1) :1–21, 2017. doi : 10.1109/TRO.2016.2597321.

- [10] Holybro. X500 V2 development kit. <https://holybro.com/products/x500-v2-kit>, 2024. Cadre 500mm, compatible PX4.
- [11] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1) :35–45, 1960.
- [12] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. *Proceedings of Imaging Understanding Workshop*, pages 121–130, 1981.
- [13] Luxonis. DepthAI : Spatial ai platform documentation. <https://docs.luxonis.com/>, 2024. Consulté le 15 janvier 2026.
- [14] Luxonis. OAK-D Pro : Product specifications. <https://docs.luxonis.com/hardware/products/OAK-D%20Pro>, 2024. Consulté le 15 janvier 2026.
- [15] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2 : Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022. doi : 10.1126/scirobotics.abm6074.
- [16] Vishwas Madyastha, Vaibhav Ravindra, Srikanth Mallikarjunan, and Anup Goyal. Extended kalman filter vs. error state kalman filter for aircraft attitude estimation. *AIAA Guidance, Navigation, and Control Conference*, 2011.
- [17] Robert Mahony, Vijay Kumar, and Peter Corke. Multirotor aerial vehicles : Modeling, estimation, and control of quadrotor. *IEEE Robotics & Automation Magazine*, 19(3) :20–32, 2012. doi : 10.1109/MRA.2012.2206474.
- [18] Raman K. Mehra. On the identification of variances and adaptive Kalman filtering. *IEEE Transactions on Automatic Control*, 15(2) :175–184, 1970. doi : 10.1109/TAC.1970.1099422.
- [19] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4 : A node-based multithreaded open source robotics framework for deeply embedded platforms. In *IEEE International Conference on Robotics and Automation*, 2015. doi : 10.1109/ICRA.2015.7140074.
- [20] A. H. Mohamed and K. P. Schwarz. Adaptive Kalman filtering for INS/GPS. *Journal of Geodesy*, 73(4) :193–203, 1999. doi : 10.1007/s001900050236.
- [21] Anastasios I. Mourikis and Stergios I. Roumeliotis. A multi-state constraint kalman filter for vision-aided inertial navigation. *Proceedings of the IEEE International Conference on Robotics and Automation*, pages 3565–3572, 2007. doi : 10.1109/ROBOT.2007.364024.

- [22] Raúl Mur-Artal, J. M. M. Montiel, and Juan D. Tardós. ORB-SLAM : A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5) : 1147–1163, 2015. doi : 10.1109/TRO.2015.2463671.
- [23] David Nistér. An efficient solution to the five-point relative pose problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(6) :756–770, 2004. doi : 10.1109/TPAMI.2004.17.
- [24] Takafumi Niwa, Jumpei Shimamura, Anton Martinsson, and Shaojie Shen. VIMO : Simultaneous visual inertial model-based odometry and force estimation. In *IEEE Robotics and Automation Letters*, 2019. doi : 10.1109/LRA.2019.2927948.
- [25] Open Robotics and PX4 Dev Team. MAVROS : Mavlink to ros gateway. <https://github.com/mavlink/mavros>, 2024. Version ROS 2 Jazzy.
- [26] Gerardo Pardo-Castellote. OMG data-distribution service : Architectural overview. In *23rd International Conference on Distributed Computing Systems Workshops*, pages 200–206, 2003. doi : 10.1109/ICDCSW.2003.1203555.
- [27] Tong Qin, Peiliang Li, and Shaojie Shen. VINS-Mono : A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4) : 1004–1020, 2018. doi : 10.1109/TRO.2018.2853729.
- [28] Royal Military Academy of Belgium. Projet GRAND : Gps-denied resilient autonomous navigation for defence. Programme de recherche RMA/VUB, 2024. Collaboration Académie Royale Militaire – Vrije Universiteit Brussel.
- [29] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. ORB : An efficient alternative to SIFT or SURF. In *IEEE International Conference on Computer Vision*, pages 2564–2571, 2011. doi : 10.1109/ICCV.2011.6126544.
- [30] Davide Scaramuzza and Friedrich Fraundorfer. Visual odometry [tutorial]. *IEEE Robotics & Automation Magazine*, 18(4) :80–92, 2011. doi : 10.1109/MRA.2011.943233.
- [31] Dan Simon. *Optimal State Estimation : Kalman, H-infinity, and Nonlinear Approaches*. John Wiley & Sons, 2006. ISBN 978-0471708582.
- [32] Joan Solà. Quaternion kinematics for the error-state Kalman filter. Technical report, Institut de Robòtica i Informàtica Industrial, 2017. arXiv :1711.02508.
- [33] Nikolas Trawny and Stergios I. Roumeliotis. Indirect kalman filter for 3d attitude estimation. Technical Report 2005-002, University of Minnesota, Dept. of Computer Science and Engineering, 2005.

-
- [34] Vladyslav Usenko, Nikolaus Demmel, David Schubert, Jörg Stückler, and Daniel Cremers. Visual-inertial mapping with non-linear factor recovery. *IEEE Robotics and Automation Letters*, 5(2) :422–429, 2020. doi : 10.1109/LRA.2019.2961227.

Annexe A

Extraits de code source

Cette annexe présente les extraits les plus significatifs du code développé. Le code complet est disponible dans `~/drone_ws/` (4 packages ROS 2).

A.1 Nœud EKF adaptatif — initialisation

```
1 class AdaptiveEKFNode(Node):
2     def __init__(self):
3         super().__init__('adaptive_ekf_node')
4
5         self.declare_parameter('vision_base_noise', 0.01) # m
6             (bonne lumiere)
7         self.declare_parameter('vision_degraded_noise', 5.0) # m
8             (obscurite)
9         self.declare_parameter('quality_threshold_good', 0.6) #
10             VIO pur si Q >= seuil
11         self.declare_parameter('quality_threshold_bad', 0.2) #
12             DR pur si Q <= seuil
13         self.declare_parameter('publish_rate_hz', 50.0)
14         self.declare_parameter('velocity_damping', 0.92)
15         self.declare_parameter('vio_stale_timeout_s', 2.0)
16         self.declare_parameter('static_calib_samples', 200) #
17             ~2s @ 100 Hz
18         self.declare_parameter('bias_ema_alpha', 0.0006)
```

Listing A.1 – Paramètres ROS 2 du nœud EKF adaptatif

A.2 Modulation adaptative de R

```

1 def compute_R_vio(self, quality_score: float) -> np.ndarray:
2     """
3     Module la matrice de covariance VIO selon le score de qualite
4     image.
5     quality_score in [0, 1]: 0 = obscurite, 1 = conditions ideales
6     """
7
8     q = np.clip(quality_score, 0.0, 1.0)
9
10
11     sigma_min = self.noise_base          # 0.01 m (bonne lumiere)
12     sigma_max = self.noise_degraded     # 5.0 m (obscurite)
13
14     # Interpolation inverse : Q=1 -> sigma=sigma_min, Q=0 -> sigma=
15     sigma_max
16     sigma = sigma_max - (sigma_max - sigma_min) * q
17     r_pos  = sigma ** 2                  # variance position
18     r_att  = (0.1 * sigma) ** 2         # variance attitude (moins
19     sensible)
20
21     R = np.zeros((7, 7))
22     R[0:3, 0:3] = r_pos * np.eye(3)     # position x, y, z
23     R[3:7, 3:7] = r_att * np.eye(4)     # quaternion
24     return R

```

Listing A.2 – Calcul de R_{VIO} en fonction du score Q

A.3 Nœud de qualité d'image — score composite

```

1 def compute_quality_score(self, img: np.ndarray) -> float:
2     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) if img.ndim==3
3     else img
4
5     # 1. Luminosite
6     mean_intensity = gray.mean() / 255.0
7     L = 1.0 - abs(mean_intensity - 0.5) * 2 # pic a 0.5
8
9     # 2. Nettete / variance du Laplacien
10    lap_var = cv2.Laplacian(gray, cv2.CV_64F).var()
11    S = min(lap_var / 500.0, 1.0)           # norme par seuil
12    empirique
13
14    # 3. Densite de features ORB
15    kp = self.orb.detect(gray, None)

```

```

14     D = min(len(kp) / 500.0, 1.0)
15
16     # 4. Contraste (ecart-type des pixels)
17     C = min(gray.std() / 80.0, 1.0)
18
19     raw_score = 0.30*L + 0.30*S + 0.25*D + 0.15*C
20
21     # Filtrage EMA (alpha=0.3) pour lisser le bruit image-a-image
22     self.ema_score = (1 - self.alpha) * self.ema_score + self.alpha
23     * raw_score
24     return float(np.clip(self.ema_score, 0.0, 1.0))

```

Listing A.3 – Calcul du score de qualité composite

A.4 Script de synchronisation capteurs

```

1 def cam_cb(self, msg):
2     t_cam = msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9
3     self.cam_stamps.append(t_cam)
4     if self.imu_stamps:
5         nearest_imu = min(self.imu_stamps, key=lambda t: abs(t -
6             t_cam))
7         self.sync_deltas.append(abs(t_cam - nearest_imu) * 1e3) #
8             ms
9
10 def print_stats(self):
11     if len(self.sync_deltas) > 10:
12         arr = list(self.sync_deltas)
13         self.get_logger().info(
14             f'cam-IMU: {np.mean(arr):.2f} +/- {np.std(arr):.2f} ms'
15             f' cam:{len(self.cam_stamps)}/s imu:{len(self.
16                 imu_stamps)}/s'
17         )

```

Listing A.4 – Mesure du délai cam-IMU (check_sensors_sync.py)

A.5 Pont MAVLink série/UDP

```

1 def forward_serial_to_udp(serial_conn, udp_sock, udp_addr):
2     """Lit les trames MAVLink sur serie et les relaie en UDP."""
3     while True:

```

```
4         data = serial_conn.read(512)
5         if data:
6             udp_sock.sendto(data, udp_addr)
7
8     def forward_udp_to_serial(udp_sock, serial_conn):
9         """Relaie les commandes UDP (MAVROS PC) vers le FCU serie."""
10        while True:
11            data, _ = udp_sock.recvfrom(4096)
12            if data:
13                serial_conn.write(data)
```

Listing A.5 – Pont MAVLink bidirectionnel (mavlink_bridge.py)

Annexe B

Paramètres et configurations

Cette annexe regroupe les paramètres clés utilisés dans la version finale du système. Un paramètre mal réglé peut transformer un vol stable en une expérience désagréable — chaque valeur ici a été validée expérimentalement.

B.1 Paramètres PX4 modifiés

Table B.1 – Paramètres PX4 modifiés pour la navigation GPS-denied (Pix32 v6C, PX4 v1.15).

Paramètre	Valeur	Défaut	Rôle
EKF2_EV_CTRL	15	14	Bitmask vision : active les 4 canaux (0b1111). Valeur 14 (0b1110) désactivait silencieusement la position horizontale — bug bloquant.
EKF2_HGT_REF	3	0	Référence de hauteur = vision externe (0=baro)
EKF2_EVP_GATE	100	5	Gate d'innovation vision (élargi pour accepter la VIO au démarrage)
EKF2_EV_DELAY	50.0	175	Délai vision mesuré : 48 ms (arrondi à 50)
SYS_HAS_GPS	0	1	Indique l'absence de module GPS
COM_ARM_WO_GPS	1	0	Autorise l'armement sans GPS
COM_RC_IN_MODE	4	0	Aucune télécommande RC requise
NAV_RCL_ACT	0	2	Pas d'action en perte RC (RTL désactivé)

Le paramètre critique est EKF2_EV_CTRL = 15 (0b1111). Il est resté à 14 pendant plusieurs jours : le bit 0 (position horizontale) était désactivé, ce qui empêchait le flag `pos_horiz_rel` de passer à `true`, et donc l'armement. Aucun message d'erreur ne le signalait.

B.2 Paramètres EKF adaptatif

Table B.2 – Paramètres principaux du nœud `adaptive_ekf_node`.

Paramètre ROS 2	Valeur	Description
<code>vision_base_noise</code>	0.01 m	$\sigma_{\min}$: bruit VIO en bonne lumière ($Q = 1$)
<code>vision_degraded_noise</code>	5.0 m	$\sigma_{\max}$: bruit VIO en obscurité ($Q = 0$)
<code>quality_threshold_good</code>	0.6	$Q \geq 0,6$: fusion VIO complète
<code>quality_threshold_bad</code>	0.2	$Q \leq 0,2$: dead reckoning pur
<code>publish_rate_hz</code>	50.0	Fréquence de publication de la pose fusionnée
<code>velocity_damping</code>	0.92	Facteur λ de freinage inertiel
<code>vio_stale_timeout_s</code>	2.0 s	Basculement DR si VIO muet depuis 2 s
<code>static_calib_samples</code>	200	Échantillons de calibration biais au démarrage (2 s)
<code>bias_ema_alpha</code>	0.0006	Taux d'apprentissage EMA biais long terme

B.3 Topics enregistrés (rosvbag)

Le script `vimo v5.0` enregistre systématiquement 38 topics. Les principaux :

Table B.3 – Topics rosvbag principaux (38 au total).

Topic	Hz	Description
<code>/mavros/imu/data</code>	100	IMU brute Pix32
<code>/mavros/state</code>	1	État FCU (armé, mode)
<code>/mavros/vision_pose/pose_cov</code>	30	Pose VIO envoyée à PX4
<code>/drone/ekf_odom</code>	50	Odométrie EKF fusionnée
<code>/drone/image_quality_score</code>	30	Score $Q \in [0, 1]$
<code>/drone/camera/image_raw</code>	7.5	Image RGB brute (local)
<code>/drone/imu/filtered</code>	100	IMU filtrée (passe-bas)
<code>/drone/safety_status</code>	2	État sécurité
<code>/mavros/battery</code>	1	Tension batterie

Les images (`image_raw`) sont exclues du profil nominal pour limiter le volume. Sans images, le débit est d'environ 5 Mo/min. Avec images (profil debug), on atteint 250 Mo/min. La session longue durée du 7 avril 2026 (1 h45 sans fuite mémoire) utilisait le profil nominal.

Annexe C

Présentation de l'institution d'accueil

Cette annexe présente l'institution d'accueil du stage, conformément aux exigences du règlement TFE de l'ISIB.

C.1 Académie Royale Militaire (RMA)

L'Académie Royale Militaire de Belgique (RMA, *Koninklijke Militaire School*) est un établissement d'enseignement supérieur et de recherche fondé en 1834, situé avenue de la Renaissance 30 à Bruxelles (Etterbeek). Elle forme les officiers de la Défense belge et accueille des chercheurs civils dans ses laboratoires.

Le département de Mécanique (*Department of Mechanical Engineering*) est organisé en trois unités de recherche :

- **Unité Robotique et Systèmes Autonomes** — développement de plateformes aériennes et terrestres autonomes, navigation GPS-denied, intelligence artificielle embarquée ;
- **Unité Matériaux et Structures** — caractérisation mécanique, résistance aux impacts, blindage ;
- **Unité Énergies et Propulsion** — systèmes de propulsion, énergétique.

Mon stage s'est déroulé au sein de l'unité Robotique et Systèmes Autonomes, sous la supervision de Joseph MIMASSI, chercheur-doctorant spécialisé en navigation autonome de drones.

C.2 Vrije Universiteit Brussel (VUB)

La VUB est une université néerlandophone située à Bruxelles (campus Etterbeek et Jette). Le département MECH (*Department of Mechanical Engineering*) de la faculté

d'Ingénierie est impliqué dans le projet GRAND via ses compétences en robotique mobile et en perception multi-capteurs.

La collaboration RMA–VUB est formalisée par un accord-cadre de recherche qui permet le partage de matériel, de laboratoires et de données. Les tests matériels de ce TFE ont été réalisés dans les locaux de la RMA, avec du matériel financé conjointement.

C.3 Le projet GRAND

Le projet **GRAND** (*GPS-denied Resilient Autonomous Navigation for Defence*) [28] est un programme de recherche pluriannuel (2023–2027) financé par la Défense belge. Ses objectifs sont :

1. Développer des algorithmes de navigation autonome robustes en environnement GPS-denied (intérieur, sous couvert forestier, brouillage électronique).
2. Concevoir des architectures de fusion multi-capteurs embarquées sur micro-drones (< 2 kg).
3. Valider les solutions sur des plateformes réelles lors d'exercices de terrain.
4. Former une nouvelle génération de chercheurs en robotique autonome via l'accueil de stagiaires et de doctorants.

Mon TFE s'inscrit dans l'objectif 2 : la conception d'un pipeline de fusion VIO/IMU adaptatif embarqué sur une plateforme X500 V6. Le matériel utilisé (châssis Holybro, contrôleur Pix32 v6C, caméra OAK-D Pro, ordinateur compagnon UP4000) est fourni par le budget du projet GRAND.

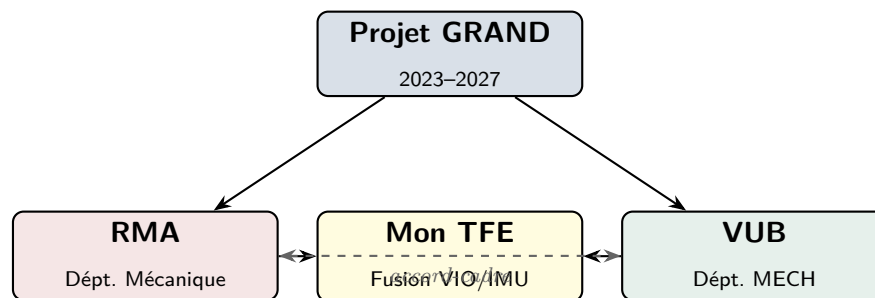


Figure C.1 – Positionnement du TFE dans l'organigramme du projet GRAND (RMA/VUB).

C.4 Accès et infrastructure

Pendant la durée du stage (septembre 2025 – mai 2026), j'ai eu accès :

- au laboratoire de robotique de la RMA (espace de vol intérieur équipé de filets de sécurité, alimentation stabilisée, réseau WiFi dédié) ;
- au matériel du projet GRAND (drone X500 V6 complet, batteries LiPo, outillage de soudure, analyseur logique) ;

- aux serveurs de calcul de la VUB pour les simulations lourdes (non utilisés dans ce TFE, car le pipeline tourne entièrement sur matériel embarqué) ;
- à la documentation interne du projet et aux réunions bimensuelles de suivi.